图论（上册）

Morning Flower and Evening Oath

目录

第一章 图论相关概念 **1**

1.1 函数图的入环链与到达查询 . . . 1

1.2 杭州 2023 H：随机事件顺序与函数图 . . . 2

第二章 树上问题 **3**

2.1 树的直径 . . . 3

2.1.1 边权修改动态树直径 . . . 3

2.1.2 树上点集直径摘要 . . . 7

2.1.3 非负权树的点集直径摘要 . . . 9

2.2 最近公共祖先 . . . 10

2.2.1 两条树路径的交集 . . . 10

2.2.2 倍增 LCA、祖先与最大边权 . . . 11

2.2.3 树路径交：投影与单点交 . . . 14

2.3 树链剖分 . . . 14

2.3.1 长链剖分与常数时间 K 级祖先 . . . 14

2.3.2 重链剖分与 LCA . . . 18

2.4 树上启发式合并 . . . 23

2.4.1 树上启发式合并：子树众数颜色和 . . . 23

2.5 虚树 . . . 26

2.5.1 虚树：关键点压缩与路径信息 . . . 26

2.6 树分治 . . . 29

2.6.1 点分治：树上距离点对计数 . . . 29

2.6.2 点分治、启发式合并与虚树用法 . . . 33

2.7 动态树分治 . . . 33

2.7.1 动态点分树：点权赋值与距离球求和 . . . 33

2.7.2 动态点分树：有符号边权活动点直径与端点 . . . 38

2.8 树哈希 . . . 43

2.8.1 确定性有根与无根树同构 . . . 43

2.8.2 双 64 位随机树哈希（Mix13） . . . 47

2.8.3 树哈希 O(n) 换根 . . . 49

第三章 最短路问题 **50**

3.1 最短路 . . . 50

3.1.1 非负权最短路 . . . 50

3.1.2 Bellman–Ford：负环影响与路径方案 . . . 51

3.1.3 SPFA：队列松弛、负环判定与有限路径 . . . 54

3.1.4 Floyd 全源最短路与原边方案 . . . 56

3.1.5 杭州 2023 G：开放时间的单位边最短路 . . . 58

3.2 k 短路 . . . 59

3.2.1 非负权严格次短行走 . . . 59

3.2.2 正权前 K 短行走 . . . 60

第四章 生成树问题 **63**
4.1 最小生成树 . . . 63
4.1.1 Kruskal 重构森林与阈值连通块 . . . 63
4.1.2 曼哈顿最小生成树 . . . 67
4.1.3 严格与非严格次小生成树、换边方案 . . . 69
4.1.4 Kruskal 最小生成树与生成森林 . . . 74
4.1.5 稠密图 Prim 最小生成森林 . . . 77
4.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明 . . . 79
4.1.7 严格次小生成树与换边 . . . 79
4.2 最小树形图 . . . 80
4.2.1 有向最小树形图 . . . 80

第五章 斯坦纳树 **84**
5.1 最小斯坦纳树与原边方案 . . . 84

第六章 连通性相关 **90**
6.1 强连通分量 . . . 90
6.1.1 函数图分解、跳转与到达 . . . 90
6.1.2 Tarjan 强连通分量与缩点 . . . 94
6.1.3 SCC 模板题与速度榜口径 . . . 96
6.2 双连通分量 . . . 97
6.2.1 点双与边双：共享 lowlink 核心 . . . 97
6.2.2 边双缩点森林（保留桥编号） . . . 101
6.3 圆方树 . . . 101
6.3.1 静态仙人掌森林最短路 . . . 101
6.3.2 由点双构造圆方森林 . . . 106

第七章 环计数问题 **108**
7.1 简单无向图四元环计数 . . . 108
7.2 无向简单图三元环枚举 . . . 110

第八章 最小环 **112**
8.1 有向图最小平均权环：线性空间 Karp . . . 112
8.2 非负权无向图最小环与原边方案 . . . 114

第九章 2-SAT **119**
9.1 2-SAT 与任意可行方案 . . . 119
9.2 2-SAT 编号方向与双连通定义 . . . 120

第十章 欧拉图 **121**
10.1 有向图字典序最小欧拉路 . . . 121
10.2 无向图欧拉路与边方向 . . . 125
10.3 欧拉回路与指定端点定向 . . . 128
10.4 任意起终点的混合欧拉路径 . . . 130

第十一章 网络流 **131**
11.1 最大流 . . . 131
11.1.1 HLPP 最高标号预流推进与流方案 . . . 131
11.1.2 最大流与方案 . . . 137
11.1.3 排列约束与最大流单位边分类 . . . 141

11.2 最小割 . . . 142
11.2.1 最大权闭合子图与选点方案 . . . 142
11.2.2 正容量最小割的可行边与必选边 . . . 143
11.2.3 Gomory–Hu 最小割树 . . . 144
11.2.4 闭合集、树形图与全局割的完整用法 . . . 145
11.3 费用流 . . . 146
11.3.1 势能最小费用最大流 . . . 146
11.3.2 允许负环的最小费用最大流 . . . 149
11.3.3 SPFA 费用流与残量方案 . . . 152
11.4 上下界网络流 . . . 153
11.4.1 上下界可行循环流 . . . 153
11.4.2 有源汇上下界最值流与边流量 . . . 154
11.5 Stoer–Wagner 算法 . . . 157
11.5.1 无向全局最小割 . . . 157

第十二章 图的匹配 160
12.1 二分图最大匹配 . . . 160
12.1.1 正则二分图完美匹配 . . . 160
12.1.2 Hopcroft–Karp 最大匹配与最小点覆盖 . . . 163
12.1.3 二分图最大匹配的可行边与必选边 . . . 166
12.1.4 二分图最大独立集方案 . . . 168
12.2 二分图最大权匹配 . . . 168
12.2.1 矩形二分图最小权匹配 . . . 168
12.3 一般图最大匹配 . . . 170
12.3.1 一般图最大基数匹配：带花树 . . . 170
12.4 一般图最大权匹配 . . . 174
12.4.1 一般图最大权匹配：带权带花树 . . . 174

第十三章 Prüfer 序列 185
13.1 Prüfer 双射与有标号树计数 . . . 185
13.1.1 双射为何成立 . . . 185
13.1.2 Cayley、指定度数与叶子数 . . . 185
13.1.3 加权 Cayley 与连接已有连通块 . . . 186

第十四章 矩阵树定理 187
14.1 Tutte 多项式求值 . . . 187
14.2 任意正模数带权矩阵树 . . . 192
14.3 带权矩阵树定理与方向约定 . . . 194
14.4 任意正模数的矩阵树计数 . . . 195

第十五章 LGV 引理 196
15.1 LGV：DAG 路径权值行列式 . . . 196

第十六章 支配树 198
16.1 支配树：Lengauer–Tarjan . . . 198

第一部分 跨类依赖 **202**

附录 A 数论 **203**
A.1 模算术简介 . . . 203
A.1.1 静态模整数 . . . 203
A.2 卢卡斯定理 . . . 207
A.2.1 Lucas：小素数模的大下标组合数 . . . 207

附录 B 组合数学 **209**
B.1 排列组合 . . . 209
B.1.1 阶乘组合数 . . . 209

附录 C 线性代数 **211**
C.1 行列式 . . . 211
C.1.1 任意正模数行列式：欧几里得消元 . . . 211

附录 D 并查集 **213**
D.1 并查集 . . . 213
D.1.1 并查集 . . . 213

附录 E 树状数组 **215**
E.1 树状数组第 k 小与组合依赖 . . . 215

附录 F 线段树 **217**
F.1 线段树基础 . . . 217
F.1.1 通用线段树与左右边界搜索 . . . 217

第一章　图论相关概念

1.1　函数图的入环链与到达查询

代码见第 90 页。接口 0-based：题目 1-based 时构造减 1，只有跳转返回的顶点加 1；最少步数的 0 与不可达的 -1 原样输出。

每个弱连通分量是一个环加若干指向环的树。入度剥离后剩下环点；反向处理剥离序列即可得到环编号、入环点和入环距离。跳转 k 步：k 小于入环距离时倍增，否则先到入环点，余步对环长取模；k 可达无符号 64 位上界，倍增表 $O(\log n)$ 层。u 到 v 最少步数：环编号不同则不可达；v 在树上时要求 $\mathrm{depth}(u) \geq \mathrm{depth}(v)$ 且跳深度差恰到 v；v 在环上时为 u 的入环步数加环上顺向距离。不可达返回 -1，自身为 0。position 只对环点有意义，树点为 -1。

1.2 杭州 2023 H：随机事件顺序与函数图

QOJ8237。事件 u 的目标 $v = b_u$：$a_u < a_v$ 必成功，$a_u \geq a_v + w_v$ 必失败，否则 u 成功当且仅当 v 更早执行且成功。确定事件的后继改为自身。到达确定失败终点或纯依赖环（含自环）则概率为零；到达确定成功终点、入环距离 d 时概率为 $1/(d+1)!$，期望 $a_u + w_u/(d+1)!$，模 10^9+7。依赖不可当作独立的 $1/2$。$\sum n \leq 500000$，阶乘表跨组只扩不缩；用通用函数图时 $O(n \log n)$ 时间与空间。

第二章　树上问题

2.1　树的直径

相关应用：固定拓扑下修改非负边权，递归欧拉序与区间加维护动态直径。

2.1.1　边权修改动态树直径

问题描述：给定一棵非负边权树，支持修改某条边的权值，并在每次修改后查询树的直径（最长路径的长度）。

核心思路：DFS 进入点时以及每个孩子返回后各记一次当前点的根距离，得到长 $2n-1$ 的序列 d。对 $i \le j \le k$，$d_i - 2d_j + d_k$ 的最大值就是直径：j 取在两点的 LCA 处时恰为路径长。线段树节点维护 `high`$=\max d$、`low`$=\max(-2d)$、`left`$=\max(d_i-2d_j)$、`right`$=\max(-2d_j+d_k)$ 和三项最大值 `best`，`merge` 枚举跨左右儿子的拼法。修改边权相当于把下端点 `child[id]` 的子树在序列中的区间 `first..last` 整体加 Δ，懒标记下这五个量分别变化 $+\Delta, -2\Delta, -\Delta, -\Delta, 0$。

接口契约：DynamicDiameter(n)，n≥1，点号 1..n。add(u,v,w) 返回从 0 开始的原边编号；输入须恰为一棵非负权无向树。build(root=1) 预处理；update(id,w) 设置原边权值；diameter() 返回最长路径长度，单点树为 0。仅返回长度，不支持 link/cut 和负权。可换根重建、复制；所有内部成员仅供只读。全部边权之和始终 ≤LLONG_MAX/4，保证中间值安全。

递归 DFS 记录进入顶点及每次孩子返回后的根距离，得到 2n-1 项欧拉序。修改父边等价于对子树首次至最后出现的闭区间加 Δ。维护 max(d[i]-2d[j]+d[k])，i≤j≤k；非负权下区间最小根距离等于 LCA 根距离，故最大值即直径。线段树五项 high、low、left、right、best 分别表示 d、-2d、前两项、后两项、完整三项最大值。统一加 Δ 后依次加 Δ、-2Δ、-Δ、-Δ、0。建树 O(n)，修改 O(log n)，查询 O(1)，空间 O(n)。DFS 保持递归，深链须给足原生栈。LOJ3163 已满分通过，排名未核验。

字段、加边与五项合并：

```
struct DynamicDiameter
{
    using ll = long long;
    struct Node
    {
        ll high, low, left, right, best, lazy;
    };
    struct Edge
    {
        int u, v;
        ll w;
    };
    int n, size = 0;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;
    vector<int> first, last, child;
    vector<Node> t;

    explicit DynamicDiameter(int n) : n(n), g(n + 1), first(n + 1), last(n + 1)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(w >= 0 && w <= LLONG_MAX / 4 - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        size = 0;
        return id;
    }
    static Node merge(const Node &a, const Node &b)
    {
        Node c;
        c.high = max(a.high, b.high);
        c.low = max(a.low, b.low);
        c.left = max({a.left, b.left, a.high + b.low});
        c.right = max({a.right, b.right, a.low + b.high});
        c.best = max({a.best, b.best, a.left + b.high, a.high + b.right});
        c.lazy = 0;
        return c;
    }
```

递归欧拉遍历与建树（接上页结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n && (int)edges.size() == n - 1);
        vector<ll> d;
        child.resize(edges.size());
        auto dfs = [&](auto &&self, int u, int parent, ll depth) -> void
        {
            first[u] = d.size();
            d.push_back(depth);
            for (auto [v, id] : g[u])
            {
                if (v == parent) continue;
                child[id] = v;
                self(self, v, u, depth + edges[id].w);
                d.push_back(depth);
            }
            last[u] = d.size() - 1;
        };
        dfs(dfs, root, 0, 0);
        size = d.size();
        t.resize(4 * size);
        auto init = [&](auto &&self, int x, int l, int r) -> void
        {
            if (l == r)
            {
                t[x] = {d[l], -2 * d[l], -d[l], -d[l], 0, 0};
                return;
            }
            int mid = (l + r) / 2;
            self(self, x * 2, l, mid);
            self(self, x * 2 + 1, mid + 1, r);
            t[x] = merge(t[x * 2], t[x * 2 + 1]);
        };
        init(init, 1, 0, size - 1);
    }
```

区间加与原边修改（接上页结构体）：

```
    void apply(int x, ll delta)
    {
        t[x].high += delta;
        t[x].low -= 2 * delta;
        t[x].left -= delta;
        t[x].right -= delta;
        t[x].lazy += delta;
    }
    void change(int x, int l, int r, int ql, int qr, ll delta)
    {
        if (ql <= l && r <= qr)
        {
            apply(x, delta);
            return;
        }
        apply(x * 2, t[x].lazy);
        apply(x * 2 + 1, t[x].lazy);
        t[x].lazy = 0;
        int mid = (l + r) / 2;
        if (ql <= mid) change(x * 2, l, mid, ql, qr, delta);
        if (qr > mid) change(x * 2 + 1, mid + 1, r, ql, qr, delta);
        t[x] = merge(t[x * 2], t[x * 2 + 1]);
    }
    void update(int id, ll w)
    {
        assert(size && 0 <= id && id < (int)edges.size());
        assert(w >= 0 && w <= LLONG_MAX / 4 - (total - edges[id].w));
        ll delta = w - edges[id].w;
        total += delta;
        edges[id].w = w;
        int u = child[id];
        change(1, 0, size - 1, first[u], last[u], delta);
    }
    ll diameter() const
    {
        assert(size);
        return t[1].best;
    }
};
```

模板题使用：CEOI2019 Dynamic Diameter（LOJ3163 待复核提交）

题意：每次修改一条边后输出直径；用上次答案解码边号和新权，边编号 0-based。

所需模板：DynamicDiameter（第 3 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://loj.ac/p/3163

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    long long w;
    cin >> n >> q >> w;
    DynamicDiameter tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long c;
        cin >> u >> v >> c;
        tree.add(u, v, c);
    }
    tree.build();
    long long last = 0;
    while (q--)
    {
        long long d, e;
        cin >> d >> e;
        int id = (d + last) % (n - 1);
        tree.update(id, (e + last) % w);
        last = tree.diameter();
        cout << last << '\n';
    }
}
```

2.1.2 树上点集直径摘要

功能：维护一棵固定非负权树上某个点集的“直径摘要”：支持插入一个点、合并两个点集，并查询点集内距离最远的一对点（直径端点 a、b 与长度），以及点集中离给定点 v 最远的点。

核心思路：关键性质：非负权树上，点集 S 的直径端点为 a, b 时，$S \cup \{v\}$ 的直径端点可在 $\{a, b, v\}$ 中取到，任意点到 S 中最远点也可取 a 或 b。因此插入 v 只需算 $\mathrm{dist}(a, v)$、$\mathrm{dist}(b, v)$，若更长就用 v 替换对应端点；合并两个集合等于把对方的两个端点依次插入；最远点查询比较到 a、b 的距离。距离由调用方的回调给出。

接口契约：默认空点集，a=b=-1、length=0。insert(v,distance) 插入顶点，重复插入合法；merge(other,distance) 合并两个点集摘要，允许自合并；farthest(v,distance) 返回 optional<pair<int,long long»（最远顶点、距离），空集返回 nullopt，距离并列时可返回任意最远点。distance(u,v) 回调必须精确返回同一棵固定非负权树的距离，点号为非负整数；所有距离 ≤LLONG_MAX。每次插入或最远查询最多两次距离调用，合并最多四次，摘要只占 O(1) 空间。保存 a、b 为点集中某对直径端点，不保留全部点；字段只读，不能用一般图度量或负权树代替。复制摘要即可建立前缀、后缀或线段树聚合，重置用默认构造。

```cpp
struct TreeDiameter
{
    using ll = long long;
    int a = -1, b = -1;
    ll length = 0;

    // distance is the metric of one fixed tree with nonnegative edge weights.
    template <class Distance> void insert(int v, const Distance &distance)
    {
        assert(v >= 0);
        if (a == -1)
        {
            a = v;
            b = v;
            return;
        }
        ll x = distance(a, v), y = distance(b, v);
        if (x >= y && x > length)
        {
            b = v;
            length = x;
        }
        else if (y > length)
        {
            a = v;
            length = y;
        }
    }
```

合并与最远点查询（接上页同一结构体）：

```
    template <class Distance> void merge(TreeDiameter other, const Distance &distance)
    {
        if (other.a == -1) return;
        insert(other.a, distance);
        insert(other.b, distance);
    }

    template <class Distance>
    optional<pair<int, ll>> farthest(int v, const Distance &distance) const
    {
        if (a == -1) return nullopt;
        ll x = distance(a, v), y = distance(b, v);
        return x >= y ? make_pair(a, x) : make_pair(b, y);
    }
};
```

2.1.3 非负权树的点集直径摘要

代码见第 7 页。非空点集只需保存一对直径端点 a, b，任意点 x 到点集的最远距离为 $\max(d(x,a), d(x,b))$。要求边权非负（可为零），不能推广到一般图或负权树。插入新点只比较到两个旧端点的距离；合并点集只插入另一摘要的两个端点。并列直径任选，端点不固定。带权距离可用 $(D_u - D_z) + (D_v - D_z)$（$z$ 为 LCA）；所有路径权和须在 long long 内。

离线添叶仍是固定树度量。 CF379F：先建出最终树再预处理距离，按出现顺序向摘要插点（初始四点也插入）。$q \le 500000$ 时 $4 + 2q$ 个点，HLD $O(q)$ 空间、$O(q \log q)$ 时间。

QOJ8235（点权互异）：令 g 为全树点权的 mex，按点权 $0, \ldots, g-1$ 建前缀摘要；某前缀全在以 x 为中心、半径 k 的范围内当且仅当其最远距离 $\le k$，二分最长前缀；$g = 0$ 时为零。边带权，不能用边数距离。

2.2 最近公共祖先

2.2.1 两条树路径的交集

问题描述：给定一棵树和两条路径 a–b、c–d，求它们的公共部分：公共部分若非空必是一条路径，返回其两个端点和公共顶点数。

核心思路：把点 x 投影到路径 a–b 上（路径上离 x 最近的点），投影就是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者。设 c,d 的投影为 p,q：若 $p\neq q$，则 c–d 必经过 p–q，这就是交集；若 $p=q$，交集至多是单点 p，用距离判断 p 是否在 c–d 上（$\mathrm{dist}(c,p)+\mathrm{dist}(p,d)=\mathrm{dist}(c,d)$），否则交集为空。距离由深度和 LCA 算出。

接口契约：path_intersection(a,b,c,d,depth,lca) 返回路径 a-b 与 c-d 的公共路径，字段 u、v 是交集端点，vertices 是公共顶点数；空交返回 {0,0,0}，单点交的 u=v。点号从 1 开始，depth 数组与 lca 回调必须来自同一棵非空树、同一个根，深度按边数计算，不能传带权根距离。支持数组或 vector 深度表，回调接收两个点并返回 LCA。将 c、d 分别投影到 a-b 得 p、q；投影是 lca(a,b)、lca(a,x)、lca(b,x) 中最深者。若 p≠q，c-d 必经过 p-q；若 p=q，检验 p 是否在 c-d 上即可区分单点交与空交。不依赖顶点编号大小。常数次 LCA 调用、O(1) 额外空间；使用 EulerLCA 时为 O(1)，HLD/倍增时为 O(log n)。公共边数为 max(0,vertices-1)。全小树根与四端点、随机换号树用路径点集独立验证；P3398 完整程序只检查是否相交，端点及数量由本地验证覆盖。

```
struct PathIntersection
{
    int u, v, vertices;
};

// Same nonempty tree and root for depth[] and lca(). Empty result is {0,0,0}.
template <class Depth, class Lca>
PathIntersection
path_intersection(int a, int b, int c, int d, const Depth &depth, Lca lca)
{
    int z = lca(a, b);
    auto project = [&](int x)
    {
        int p = z;
        for (int v : {lca(a, x), lca(b, x)})
            if (depth[v] > depth[p]) p = v;
        return p;
    };
    auto distance = [&](int u, int v)
    {
        return 1LL * depth[u] + depth[v] - 2LL * depth[lca(u, v)];
    };
    int p = project(c), q = project(d);
    if (p == q && distance(c, p) + distance(p, d) != distance(c, d)) return {0, 0, 0};
    return {p, q, int(distance(p, q) + 1)};
}
```

2.2.2 倍增 LCA、祖先与最大边权

功能：对一棵带边权的树做倍增预处理，支持：求 u 的 k 级祖先、两点的 LCA 与路径边数，以及两点之间路径上的最大边权。

核心思路：`up[k][u]` 是 u 向上 2^k 步的祖先，`mx[k][u]` 是这 2^k 条边的最大权，DFS 进入 u 时由 `up[k-1]` 两段拼出。跳 k 级祖先按 k 的二进制位逐位跳。求 LCA 时先把较深点跳到同一深度，若重合即为答案；否则从高位到低位，只要两点的 2^k 级祖先不同就同时上跳，最后父亲即 LCA。`max_edge` 沿完全相同的跳跃过程，把经过的各段 `mx` 取最大。0 号点作哨兵，表示“没有祖先”。

接口契约：构造 LiftingLCA(n)，add(u,v,w=1) 加边，build(root=1) 预处理非空无向树，点号 1..n。jump(u,k) 返回 k 级祖先，k 非负，超过根返回 0；lca(u,v) 求最近公共祖先；distance 返回路径边数，不是权值和；max_edge 返回路径最大边权，同点返回 nullopt，其他情况返回有值的 optional<long long>。支持负权和 LLONG_MIN，LOW 只是最大值合并的单位元，不表示空路径。DFS 递归，根深度为 0；up[k][u] 与 mx[k][u] 是 2 的 k 次方长度向上段的祖先与最大边权，0 号节点为哨兵。可重复 build 换根；修改图后必须重建。预处理 O(n log n) 时间与空间，各查询 O(log n)。不需要边权最值时，可删去 mx 和 max_edge 相关部分。全小树点对/根、极值权重及 50 万点负权链本地验证；P3379、POJ1330 驱动只验证 LCA，最大边权仍是本地证据。

```
struct LiftingLCA
{
    using ll = long long;
    static constexpr ll LOW = numeric_limits<ll>::lowest();
    int n, levels;
    bool ready = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mx;
    vector<int> depth;

    LiftingLCA(int n) : n(n), g(n + 1), depth(n + 1)
    {
        assert(n > 0);
        levels = 1;
        while ((1LL << levels) <= n) levels++;
        up.assign(levels, vector<int>(n + 1));
        mx.assign(levels, vector<ll>(n + 1, LOW));
    }

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
        ready = false;
    }
```

```
    void dfs(int u, int parent, ll weight)
    {
        up[0][u] = parent;
        mx[0][u] = weight;
        for (int k = 1; k < levels; k++)
        {
            int p = up[k - 1][u];
            up[k][u] = up[k - 1][p];
            mx[k][u] = max(mx[k - 1][u], mx[k - 1][p]);
        }
        for (auto [v, w] : g[u])
            if (v != parent)
            {
                depth[v] = depth[u] + 1;
                dfs(v, u, w);
            }
    }

    // Nonempty tree only. Node 0 means no ancestor.
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        depth[root] = 0;
        dfs(root, 0, LOW);
        ready = true;
    }

    int jump(int u, long long steps) const
    {
        assert(ready && 1 <= u && u <= n && steps >= 0);
        if (steps > depth[u]) return 0;
        for (int k = 0; k < levels; k++)
            if (steps >> k & 1) u = up[k][u];
        return u;
    }
```

```
    int lca(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        if (depth[u] < depth[v]) swap(u, v);
        u = jump(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int k = levels - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        return up[0][u];
    }

    int distance(int u, int v) const
    {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }

    optional<ll> max_edge(int u, int v) const
    {
        assert(ready && 1 <= u && u <= n && 1 <= v && v <= n);
        if (u == v) return nullopt;
        if (depth[u] < depth[v]) swap(u, v);
        ll answer = LOW;
        int delta = depth[u] - depth[v];
        for (int k = 0; k < levels; k++)
            if (delta >> k & 1)
            {
                answer = max(answer, mx[k][u]);
                u = up[k][u];
            }
        if (u == v) return answer;
        for (int k = levels - 1; k >= 0; k--)
            if (up[k][u] != up[k][v])
            {
                answer = max(answer, max(mx[k][u], mx[k][v]));
                u = up[k][u];
                v = up[k][v];
            }
        return max(answer, max(mx[0][u], mx[0][v]));
    }
};
```

模板题使用：Luogu P3379

题意：给定以 root 为根的树，回答两点的最近公共祖先。加完所有边后 build(root)，再逐条调用 lca；点号为 1..n。此题不涉及跳祖先或路径最大边权。

所需模板：LiftingLCA（第 11 页，2.2.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3379

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q, root;
    cin >> n >> q >> root;
    LiftingLCA graph(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        graph.add(u, v);
    }
    graph.build(root);
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << graph.lca(u, v) << '\n';
    }
    return 0;
}
```

2.2.3 树路径交：投影与单点交

接口见第 10 页。x 在路径 a–b 上的投影是 $\mathrm{lca}(a,b)$、$\mathrm{lca}(a,x)$、$\mathrm{lca}(b,x)$ 中深度最大者（同一根）。设 c,d 的投影为 p,q：$p\neq q$ 时交集为 p–q；$p=q$ 时若 $d(c,p)+d(p,d)=d(c,d)$ 则为单点交，否则为空。距离与 depth 按边数，不能换成带权根距离。返回交集端点和顶点数，空交为 $\{0,0,0\}$，公共边数 $\max(0,\text{vertices}-1)$。每次调用常数次 LCA；用 HLD 回调为 $O(\log n)$。

2.3 树链剖分

2.3.1 长链剖分与常数时间 K 级祖先

功能：对有根树做长链剖分，支持在线查询任意点 u 的 k 级祖先。

核心思路：`dfs1` 求深度、倍增表 `fa` 和子树高度 `len`，高度最大的儿子作为长儿子 `son`；`dfs2` 把树剖成长链，对每个长度为 L 的链顶 t 存下从 t 向下的整条链 `down[t]` 和从 t 向上的 L 个祖先 `up[t]`，总量 $O(n)$。查询时先用倍增跳 k 的最高二进制位 2^j 到 u'；u' 下方至少还有 2^j 层，所以它所在长链在链顶 t 之下至少延伸 $d+2^j$（d 为 u' 到 t 的深度差），而剩余步数小于 2^j。剩余 $k\le d$ 时在 `down[t]` 中取，否则在 `up[t]` 中取第 $k-d$ 个。

接口契约：LongChain(n)，1≤n<INT_MAX，点号 1..n；add(u,v) 加入无向树边，调用方保证全部边恰好构成一棵树，不支持环、重边、自环或森林。build(root=1) 以指定根重建，dep[root]=1；kth(u,k) 返回 k 级祖先，k≥0，k=0 返回自身，越过根返回 0。查询前必须 build，加边后必须重建；支持重复 build、换根与值复制。son 按子树最大高度选长儿子，不能换成子树大小；len 含自身。链顶 t 的 down[t] 从 t 向下，up[t][0]=t 且向上越根补 0；成

员只读，dfs1/dfs2 仅内部使用。预处理 O(n log(n+1))，空间 O(n log(n+1))，两次 DFS 均递归、栈深 O(n)。查询先跳最高 2 的幂，再在链顶上下表中查一次，32 位字长模型下 O(1)；k=0 必须先处理，不能求 0 的最高位。

```
struct LongChain
{
    int n;
    vector<int> dep, len, son, top;
    vector<vector<int>> g, fa, up, down;

    LongChain(int n) : n(n), g(n + 1)
    {
        assert(1 <= n && n < INT_MAX);
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[0][u] = p;
        dep[u] = dep[p] + 1;
        for (int j = 1; j < (int)fa.size(); j++)
            fa[j][u] = fa[j - 1][fa[j - 1][u]];
        for (int v : g[u])
        {
            if (v == p) continue;
            dfs1(v, u);
            if (len[v] + 1 > len[u])
            {
                len[u] = len[v] + 1;
                son[u] = v;
            }
        }
    }
```

递归划分长链并填表（接上页同一结构体）：

```
36      void dfs2(int u, int t)
37      {
38          top[u] = t;
39          if (u == t)
40          {
41              down[t].reserve(len[u]);
42              up[t].resize(len[u]);
43              int x = u;
44              for (int &v : up[t])
45              {
46                  v = x;
47                  x = fa[0][x];
48              }
49          }
50          down[t].push_back(u);
51          if (son[u]) dfs2(son[u], t);
52          for (int v : g[u])
53          {
54              if (v != fa[0][u] && v != son[u])
55                  dfs2(v, v);
56          }
57      }
```

重建与常数时间查询（接上页同一结构体）：

```
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        dep.assign(n + 1, 0);
        len.assign(n + 1, 1);
        son.assign(n + 1, 0);
        top.assign(n + 1, 0);
        fa.assign(bit_width((unsigned)n), vector<int>(n + 1));
        up.assign(n + 1, {});
        down.assign(n + 1, {});
        dfs1(root, 0);
        dfs2(root, root);
    }

    int kth(int u, int k) const
    {
        assert(1 <= u && u <= n && k >= 0 && !fa.empty());
        if (k >= dep[u]) return 0;
        if (k == 0) return u;
        int j = bit_width((unsigned)k) - 1;
        u = fa[j][u];
        k -= 1 << j;
        int t = top[u];
        int d = dep[u] - dep[t];
        if (k <= d) return down[t][d - k];
        return up[t][k - d];
    }
};
```

为什么一次跳跃就够 先向上跳 p（不超过 k 的最大 2 的幂），到达 u，剩余 r 小于 p。原查询点是 u 下方距离 p 的后代，所以 u 的子树高度至少 p+1。u 所在长链顶 t 的链长至少 dep[u]-dep[t]+p+1，足以覆盖 r；若 r 不超过 u 到 t 的距离就查 down，否则查 up。选择长儿子必须按高度，不能按子树大小。

边界与空间 up[t][0] 和 down[t][0] 都等于 t；up 越过根填 0。先处理 k=0 和越根，避免对 0 求最高位。各长链长度之和为 n，上下表各存 n 个 int；倍增表为 O(n log n)。预处理两次 DFS 都保持递归，栈深可达 n。

模板题使用：P5903 【模板】树上 K 级祖先

题意：按题意用 32 位无符号 xorshift 顺序生成询问；根深度为 1，先生成 x 再生成 k，按 64 位累计 i* 答案的异或；n≤500000，q≤5000000。

所需模板：LongChain（第 14 页，2.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5903

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    uint32_t s;
    cin >> n >> q >> s;
    LongChain tree(n);
    int root = 0;
    for (int u = 1; u <= n; u++)
    {
        int p;
        cin >> p;
        if (p) tree.add(u, p);
        else root = u;
    }
    tree.build(root);
    auto get = [&]()
    {
        s ^= s << 13;
        s ^= s >> 17;
        s ^= s << 5;
        return s;
    };
    int last = 0;
    unsigned long long ans = 0;
    for (int i = 1; i <= q; i++)
    {
        int x = (get() ^ last) % n + 1;
        int k = (get() ^ last) % tree.dep[x];
        last = tree.kth(x, k);
        ans ^= 1ULL * i * last;
    }
    cout << ans << '\n';
}
```

2.3.2 重链剖分与 LCA

功能：对一棵树做重链剖分：给出 DFS 序 `dfn`，使每棵子树、每条重链都是连续区间；支持求 LCA，以及把 u–v 路径拆成 $O(\log n)$ 个 `dfn` 区间回调（可只取边对应的位置，也可按 u 到 v 的真实顺序给出），便于套线段树做路径修改与查询。

核心思路：`dfs1` 求父亲、深度、子树大小和重儿子 `son`（子树最大的儿子）；`dfs2` 先递归重儿子，使同一重链的 `dfn` 连续，`top` 为链顶。查询时每次让链顶更深的一端整段跳过本链（上报区间 $[\mathrm{dfn}[\mathrm{top}[u]], \mathrm{dfn}[u]]$）并跳到链顶的父亲，直到两点同链，再上报二者之间的一段；边模式下边权存在深的一端，最后一段跳过较浅点即 LCA。`path_ordered` 中 u 侧的段是自下而上走的，立即以 `rev=true` 回调；v 侧的段先存入 `down`，最后逆序以正向回调。

接口契约： add(u,v) 建无向树；build(root) 后读取 fa、dep、siz、son、top、dfn、rk。子树对应 [dfn[u],dfn[u]+siz[u]-1]。lca(u,v) 为 O(log n)，path(u,v,work,edge) 分解路径；work(l,r) 仅适用于可交换聚合，边权模式排除 LCA 位置。输入必须是连通无环树，建树 O(n)，两遍递归 DFS；支持改变根后重复 build，dfn 会重新编号。path_ordered(u, v, work, edge) 的 edge 默认为 false；按真实 u 到 v 顺序回调 work(l, r, rev)：1-based 闭区间 l≤r，rev=true 按 r 到 l 访问，否则按 l 到 r。支持非交换聚合，O(log n) 段与额外空间；结合线段树通常每次查询 $O(\log^2 n)$。edge=true 仅删除 LCA 对应位置，仍须自行处理边算子的方向；不自动求逆。u=v 时点模式一次单点回调，边模式无回调；改变根后须重建依赖 dfn 的结构。

```
struct HLD
{
    int n, timer = 0;
    vector<vector<int>> g;
    vector<int> fa, dep, siz, son, top, dfn, rk;

    HLD(int n)
        : n(n),
          g(n + 1),
          fa(n + 1),
          dep(n + 1),
          siz(n + 1),
          son(n + 1),
          top(n + 1),
          dfn(n + 1),
          rk(n + 1)
    {
    }

    void add(int u, int v)
    {
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void dfs1(int u, int p)
    {
        fa[u] = p;
        siz[u] = 1;
        son[u] = 0;
        for (int v : g[u])
        {
            if (v == p) continue;
            dep[v] = dep[u] + 1;
            dfs1(v, u);
            siz[u] += siz[v];
            if (!son[u] || siz[v] > siz[son[u]]) son[u] = v;
        }
    }
```

重编号、LCA 与可交换路径（接上页同一结构体）：

```
    void dfs2(int u, int t)
    {
        top[u] = t;
        dfn[u] = ++timer;
        rk[timer] = u;
        if (son[u]) dfs2(son[u], t);
        for (int v : g[u])
        {
            if (v != fa[u] && v != son[u]) dfs2(v, v);
        }
    }

    // Connected tree only. Recursive DFS; subtree interval is [dfn, dfn+siz-1].
    void build(int root = 1)
    {
        assert(1 <= root && root <= n);
        timer = 0;
        dep[root] = 0;
        dfs1(root, root);
        dfs2(root, root);
        assert(timer == n);
    }

    int lca(int u, int v) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            u = fa[top[u]];
        }
        return dep[u] < dep[v] ? u : v;
    }

    // Commutative operations only; edge=true excludes the LCA's position.
    template <class F> void path(int u, int v, F work, bool edge = false) const
    {
        while (top[u] != top[v])
        {
            if (dep[top[u]] < dep[top[v]]) swap(u, v);
            work(dfn[top[u]], dfn[u]);
            u = fa[top[u]];
        }
        if (dep[u] > dep[v]) swap(u, v);
        if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v]);
    }
```

按真实方向分解路径（接上页同一结构体）：

```
    // Emit u-to-v order; reverse=true traverses the closed interval from r to l.
    template <class F> void path_ordered(int u, int v, F work, bool edge = false) const
    {
        vector<pair<int, int>> down;
        while (top[u] != top[v])
        {
            if (dep[top[u]] >= dep[top[v]])
            {
                work(dfn[top[u]], dfn[u], true);
                u = fa[top[u]];
            }
            else
            {
                down.emplace_back(dfn[top[v]], dfn[v]);
                v = fa[top[v]];
            }
        }
        if (dep[u] >= dep[v])
        {
            if (dfn[v] + edge <= dfn[u]) work(dfn[v] + edge, dfn[u], true);
        }
        else if (dfn[u] + edge <= dfn[v]) work(dfn[u] + edge, dfn[v], false);
        for (auto it = down.rbegin(); it != down.rend(); ++it)
            work(it->first, it->second, false);
    }
};
```

模板题使用：Library Checker Vertex Set Path Composite

题意：静态无向树，1≤N,Q≤200000。点函数 f(x)=a*x+b，对 998244353 取模；题面 1≤a,c<mod，0≤b,d,x<mod。0 p c d 改单点函数，1 u v x 按 u 到 v（含两端点）依次代入。题目点号 0-based，传 HLD 时加 1；HLD 闭区间 [l,r] 对应 segtree 半开 [l−1,r)。S 存正向与反向复合，不能把组合顺序反过来或用逆函数代替反向序列。点改 O(log N)，路径 O($\log^2$ N)，空间 O(N)；保留递归 DFS，最大链须足够大栈。

所需模板：HLD（第 18 页，2.3.2 节）、segtree（第 217 页，F.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/vertex_set_path_composite

```
using S = array<long long, 4>;
const int mod = 998244353;

S op(S a, S b)
{
    return {b[0] * a[0] % mod,
            (b[0] * a[1] + b[1]) % mod,
            a[2] * b[2] % mod,
            (a[2] * b[3] + a[3]) % mod};
}

S e()
{
    return {1, 0, 1, 0};
}

int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    vector<S> a(n), val(n);
    for (auto &x : a)
    {
        cin >> x[0] >> x[1];
        x[2] = x[0];
        x[3] = x[1];
    }
    HLD h(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        h.add(u + 1, v + 1);
    }
    h.build();
    for (int u = 1; u <= n; u++) val[h.dfn[u] - 1] = a[u - 1];
    segtree<S, op, e> s(val);
```

继续处理操作（接上页同一 main）：

```
    while (q--)
    {
        int t;
        cin >> t;
        if (t == 0)
        {
            int p;
            long long c, d;
            cin >> p >> c >> d;
            s.set(h.dfn[p + 1] - 1, {c, d, c, d});
        }
        else
        {
            int u, v;
            long long x;
            cin >> u >> v >> x;
            h.path_ordered(u + 1, v + 1, [&](int l, int r, bool rev)
            {
                S z = s.prod(l - 1, r);
                int k = rev ? 2 : 0;
                x = (z[k] * x + z[k + 1]) % mod;
            });
            cout << x << '\n';
        }
    }
}
```

2.4 树上启发式合并

2.4.1 树上启发式合并：子树众数颜色和

问题描述：给定有根树，每个点有一种颜色。对每个点 u，求其子树中出现次数最多的颜色的出现次数，以及所有达到该次数的颜色编号之和。

核心思路：树上启发式合并（dsu on tree）。`prepare` 求子树大小、重儿子和 DFS 序，使子树对应 `order` 中一段区间。`solve(u,keep)` 先处理所有轻儿子并清空它们的统计，再处理重儿子并保留其统计；接着把轻儿子子树的点按区间逐个 `insert`，再插入 u，此时全局 `freq`、最大频次 `best` 及对应颜色和 `sum` 恰为 u 子树的答案。若 u 不需保留，就把整棵子树的频次减回去并将 `best`、`sum` 归零，而不是逐个删除时维护众数。每个点只在其到根路径上的轻边处被重新插入，次数为对数级。颜色先离散化。

接口契约：SubtreeColors(n)，点号 1..n，add(u,v) 建无向树；要求图连通且无环，n=0 可空运行。run(colors,root=1) 传入长度 n+1 的 int 颜色数组，colors[0] 忽略。answer[u] 为子树内所有最高频颜色的原始编号之和，max_freq[u] 为该频次；支持负颜色与稀疏编号，内部离散化，和用 long long。重复 run 可更换颜色或根。重儿子信息保留，轻儿子通过 DFS 序区间加入；清空整棵临时子树后再重置最大值，不尝试逐次删除时维护众数。O(n log n) 时间、O(n) 空间，全部 DFS 递归。insert 是内部增量统计，可按题目修改统计量，但相应清空规则也必须调整。

```
struct SubtreeColors
{
    int n, best = 0;
    long long sum = 0;
    vector<vector<int>> g;
    vector<int> siz, heavy, tin, tout, order, color, value, freq;
    vector<int> max_freq;
    vector<long long> answer;

    SubtreeColors(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }

    void prepare(int u, int p)
    {
        siz[u] = 1;
        tin[u] = order.size();
        order.push_back(u);
        for (int v : g[u])
        {
            if (v == p) continue;
            prepare(v, u);
            siz[u] += siz[v];
            if (siz[v] > siz[heavy[u]]) heavy[u] = v;
        }
        tout[u] = (int)order.size() - 1;
    }

    void insert(int u)
    {
        int c = color[u];
        freq[c]++;
        if (freq[c] > best)
        {
            best = freq[c];
            sum = value[c];
        }
        else if (freq[c] == best)
            sum += value[c];
    }
```

启发式合并与运行入口（接上页同一结构体）：

```
    void solve(int u, int p, bool keep)
    {
        for (int v : g[u])
        {
            if (v != p && v != heavy[u]) solve(v, u, false);
        }
        if (heavy[u]) solve(heavy[u], u, true);
        for (int v : g[u])
        {
            if (v == p || v == heavy[u]) continue;
            for (int i = tin[v]; i <= tout[v]; i++) insert(order[i]);
        }
        insert(u);
        max_freq[u] = best;
        answer[u] = sum;
        if (!keep)
        {
            for (int i = tin[u]; i <= tout[u]; i++) freq[color[order[i]]]--;
            best = 0;
            sum = 0;
        }
    }

    // A connected tree, colors[1..n]. Sum all colors tied for maximum frequency.
    void run(const vector<int> &colors, int root = 1)
    {
        assert((int)colors.size() == n + 1);
        siz.assign(n + 1, 0);
        heavy = siz;
        tin = siz;
        tout = siz;
        color = siz;
        max_freq = siz;
        answer.assign(n + 1, 0);
        order.clear();
        value.assign(colors.begin() + 1, colors.end());
        sort(value.begin(), value.end());
        value.erase(unique(value.begin(), value.end()), value.end());
        freq.assign(value.size(), 0);
        best = 0;
        sum = 0;
        if (!n) return;
        assert(1 <= root && root <= n);
        for (int u = 1; u <= n; u++)
            color[u] =
                lower_bound(value.begin(), value.end(), colors[u]) - value.begin();
        prepare(root, 0);
        solve(root, 0, false);
    }
};
```

2.5 虚树

2.5.1 虚树：关键点压缩与路径信息

功能：给定一棵带非负边权的树，预处理后对每组关键点构建虚树：只保留关键点及它们两两的 LCA，原树中的一段路径压成一条虚边，并给出虚边对应的原路径长度和路径上最小边权，供在虚树上做 DP。

核心思路：预处理一遍 DFS 求入出时间 `tin`/`tout`（用于 $O(1)$ 判祖先）、根距 `dist`，以及倍增祖先 `up` 和跳 2^j 步的最小边权 `mn`。建树时把关键点按 `tin` 排序去重，补上 DFS 序中相邻两点的 LCA，再排序去重，这个点集对 LCA 封闭。然后按 DFS 序扫描，维护一条祖先链栈：弹出不是当前点祖先的栈顶，剩下的栈顶就是当前点在虚树中的父亲，连一条虚边。虚边的长度由 `dist` 相减得到，最小边权由 `path` 沿倍增上跳求出。

接口契约：VirtualTree(n)，n≥1，点号 1..n；add(u,v,w=1) 建非负权连通树，无环和重边。prepare(root=1) 递归预处理，可换根重建；根到任一点距离须在 long long 范围内。build(keys) 自动去重并补齐必要 LCA，返回 vertices 和 edges；全部编号沿用原树，vertices 按 DFS 序排列，非空时首项即虚树根。edges 为 (from,to,distance,minimum)，分别是祖先到后代的虚边、压缩路径长度和路径最小边权。空关键点返回空结果，单点返回单节点；原树根不会强制加入，需要时由调用者加入 keys。边按后代 DFS 序排列，可逆序做自底向上的 DP。不分配或清空 n 个查询邻接表；每次仅返回 O(k) 结果。预处理 O(n log n) 时间空间，单次 O(k log k+k log n)。lca、ancestor 也可直接使用，要求有效点号且已经 prepare。构造用的祖先链栈属于虚树算法，原树 DFS 保持递归。

```
struct VirtualTree
{
    using ll = long long;

    struct Edge
    {
        int from, to;
        ll distance, minimum;
    };

    struct Result
    {
        vector<int> vertices;
        vector<Edge> edges;
    };

    int n, log = 1, timer = 0;
    vector<vector<pair<int, ll>>> g;
    vector<vector<int>> up;
    vector<vector<ll>> mn;
    vector<int> depth, tin, tout;
    vector<ll> dist;

    VirtualTree(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w = 1)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

预处理与最近公共祖先（接上页同一结构体）：

```
    void dfs(int u, int p)
    {
        tin[u] = ++timer;
        up[0][u] = p;
        for (int j = 1; j < log; j++)
        {
            up[j][u] = up[j - 1][up[j - 1][u]];
            mn[j][u] = min(mn[j - 1][u], mn[j - 1][up[j - 1][u]]);
        }
        for (auto [v, w] : g[u])
        {
            if (v == p) continue;
            assert(dist[u] <= numeric_limits<ll>::max() - w);
            depth[v] = depth[u] + 1;
            dist[v] = dist[u] + w;
            mn[0][v] = w;
            dfs(v, u);
        }
        tout[u] = timer;
    }

    void prepare(int root = 1)
    {
        assert(1 <= root && root <= n);
        log = 1;
        while ((1LL << log) <= n) log++;
        up.assign(log, vector<int>(n + 1));
        mn.assign(log, vector<ll>(n + 1, numeric_limits<ll>::max()));
        depth.assign(n + 1, 0);
        tin = depth;
        tout = depth;
        dist.assign(n + 1, 0);
        timer = 0;
        dfs(root, root);
    }

    bool ancestor(int u, int v) const { return tin[u] <= tin[v] && tout[v] <= tout[u]; }

    int lca(int u, int v) const
    {
        if (ancestor(u, v)) return u;
        if (ancestor(v, u)) return v;
        for (int j = log - 1; j >= 0; j--)
        {
            if (!ancestor(up[j][u], v)) u = up[j][u];
        }
        return up[0][u];
    }
```

压缩路径与虚树构造（接上页同一结构体）：

```
    Edge path(int u, int v) const
    {
        assert(u != v && ancestor(u, v));
        ll minimum = numeric_limits<ll>::max();
        int x = v, steps = depth[v] - depth[u];
        for (int j = 0; j < log; j++)
        {
            if (steps >> j & 1)
            {
                minimum = min(minimum, mn[j][x]);
                x = up[j][x];
            }
        }
        return {u, v, dist[v] - dist[u], minimum};
    }

    // Uses original vertex IDs. Empty input gives an empty result.
    Result build(vector<int> keys) const
    {
        if (keys.empty()) return {};
        for (int u : keys) assert(1 <= u && u <= n);
        auto cmp = [&](int u, int v)
        {
            return tin[u] < tin[v];
        };
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        int k = keys.size();
        for (int i = 1; i < k; i++) keys.push_back(lca(keys[i - 1], keys[i]));
        sort(keys.begin(), keys.end(), cmp);
        keys.erase(unique(keys.begin(), keys.end()), keys.end());
        Result result{keys, {}};
        vector<int> st;
        for (int u : keys)
        {
            while (!st.empty() && !ancestor(st.back(), u)) st.pop_back();
            if (!st.empty()) result.edges.push_back(path(st.back(), u));
            st.push_back(u);
        }
        return result;
    }
};
```

2.6 树分治

2.6.1 点分治：树上距离点对计数

问题描述：给定非负边权的树（或森林）和阈值 k，统计距离不超过 k 的无序点对数，或距离恰好为 k 的点对数（等于“$\le k$”减“$\le k-1$”）。

核心思路：点分治：每次在当前连通块取重心 c，经过 c 的路径距离等于两端到 c 的距离之和。预处理时对每个重心存下块内所有点到 c 的距离 `all[c]`（含 c 自身的 0），以及删去 c 后各子块的距离 `part[c]`，都已排序；然后删去 c 递归处理各子块，`parent` 记点分树父亲。计数时 `pairs` 用双指针统计有序数组中和 $\le k$ 的对数：`all[c]` 中的对数减去每个 `part[c]` 内部的对数，就是经过 c 的路径数；对所有重心求和，每对点恰在第一个把它们分开的重心处计入一次。

接口契约：CentroidPairs(n)，点号 1..n；add(u,v,w) 建树，允许森林但禁止环、重边、自环，边权非负；所有简单路径长度须能用 long long 表示。build() 预处理，可重复调用；改变边后须重建。count_leq(k) 统计距离 ≤k 的无序不同顶点对，count_exact(k) 统计距离恰为 k 的点对；负 k 返回 0，零距离只统计不同顶点，跨连通块不配对。parent[c] 是点分树父节点，根为 0。all[c] 为该层分治块到重心的距离，含重心距离 0；part[c] 为去掉重心后各子块的距离。预处理 O(n $\log^2$ n)，空间 O(n log n)，每次计数 O(n log n)；固定树上多次查询可以复用排序结果。DFS 递归，距离和比较采用减法避免溢出。不是动态点分树更新模板。

```
struct CentroidPairs
{
    using ll = long long;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<int> siz, removed, parent;
    vector<vector<ll>> all;
    vector<vector<vector<ll>>> part;

    CentroidPairs(int n)
        : n(n),
          g(n + 1),
          siz(n + 1),
          removed(n + 1),
          parent(n + 1),
          all(n + 1),
          part(n + 1)
    {
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n && u != v && w >= 0);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }

    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }
```

重心与分解（接上页同一结构体）：

```
    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
        {
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        }
        return u;
    }

    void collect(int u, int p, ll d, vector<ll> &out)
    {
        out.push_back(d);
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            assert(d <= numeric_limits<ll>::max() - w);
            collect(v, u, d + w, out);
        }
    }

    void decompose(int entry, int p)
    {
        size_dfs(entry, 0);
        int c = centroid(entry, 0, siz[entry]);
        removed[c] = 1;
        parent[c] = p;
        all[c] = {0};
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            part[c].push_back({});
            auto &d = part[c].back();
            collect(v, c, w, d);
            sort(d.begin(), d.end());
            all[c].insert(all[c].end(), d.begin(), d.end());
        }
        sort(all[c].begin(), all[c].end());
        for (auto [v, w] : g[c])
        {
            if (!removed[v]) decompose(v, c);
        }
    }
```

重建与距离计数（接上页同一结构体）：

```
    // Input must be a forest. Rebuild after changing edges.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        for (int u = 1; u <= n; u++)
        {
            all[u].clear();
            part[u].clear();
        }
        for (int u = 1; u <= n; u++)
        {
            if (!removed[u]) decompose(u, 0);
        }
    }

    ll pairs(const vector<ll> &d, ll k) const
    {
        int l = 0, r = (int)d.size() - 1;
        ll answer = 0;
        while (l < r)
        {
            if (d[l] <= k - d[r])
            {
                answer += r - l;
                l++;
            }
            else
                r--;
        }
        return answer;
    }

    // Unordered pairs of distinct connected vertices. Nonnegative edge weights.
    ll count_leq(ll k) const
    {
        if (k < 0) return 0;
        ll answer = 0;
        for (int c = 1; c <= n; c++)
        {
            answer += pairs(all[c], k);
            for (const auto &d : part[c]) answer -= pairs(d, k);
        }
        return answer;
    }

    ll count_exact(ll k) const
    {
        if (k < 0) return 0;
        return count_leq(k) - (k ? count_leq(k - 1) : 0);
    }
};
```

模板题使用：Luogu P3806

题意：正权树上查询距离为 k 的不同顶点对是否存在。先 build 一次，再用 count_exact 的结果是否非零输出 AYE/NAY。n 至 1 万、询问至 100、边权至 1 万；该程序查询存在性，底层计数另支持零权和森林。

所需模板：CentroidPairs（第 29 页，2.6.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3806

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    CentroidPairs tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        tree.add(u, v, w);
    }
    tree.build();
    while (m--)
    {
        long long k;
        std::cin >> k;
        std::cout << (tree.count_exact(k) ? "AYE" : "NAY") << '\n';
    }
}
```

2.6.2 点分治、启发式合并与虚树用法

点分治 count_exact 计无序不同顶点对，预处理 $O(n \log^2 n)$，每次查询 $O(n \log n)$；边修改后须重建。

启发式合并：轻子树先算后清空，重子树保留；并列最高频颜色编号之和须用 long long。

虚树：显式加入根后构造，只清零本次 vertices 的 dp；逆序遍历虚边，孩子为关键点取路径最小边权，否则取它与孩子 dp 的较小值（用 minimum，不用 distance）。

2.7 动态树分治

2.7.1 动态点分树：点权赋值与距离球求和

问题描述：给定单位边权的树和点权，支持单点赋值，以及询问到点 u 距离不超过 k 的所有点的点权和（动态点分治）。

核心思路：点分治时，对重心 c 所在连通块按到 c 的距离把点权放入桶，线性建出 Fenwick `all[c]`；对 c 的每个分支 b 另建 `part[c][b]`，同样以到 c 的距离为下标。每个点在 `path` 中记下各级重心祖先 c、到 c 的距离 d 和所在分支 b。查询 u 时对每个重心祖先，加上 `all[c]` 中距离不超过 $k-d$ 的和，再减去 `part[c][b]` 中同样范围的和：与 u 同分支的点到 u 的路径不经过 c，它们会在更深的重心处被统计。赋值时把差值加到 `path` 上的所有桶中。

代码依赖：Fenwick（第 215 页，E.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：构造 CentroidSum(n)，T 默认为 long long；n≥1，输入是固定连通单位边树。点号 0..n-1；先 add(u,v)，再 build(values)。set(u,x) 是点权赋值；query(u,k) 返回到 u 距离 ≤k 的全部点权和，含 u 自身；负半径返回 0，

半径用 long long。build 可在同一拓扑上重建并替换全部权值，建树后不再 add 边，不提供 link/cut。每点保存重心祖先、距离和分支，使用总距离桶减同支桶容斥；距离 d 对应 Fenwick 下标 d+1。线性建每层 Fenwick，构造/空间 O(n log n)，修改/查询 O($\log^2$ n)。T 支持零值及加减，所有桶和、赋值差与容斥中间值均须可表示；可用负权，不使用 kth。内部图、路径、桶和辅助函数不能外部改写。原树 DFS 递归深度最坏 O(n)，不等于重心层数；最大链测试提供 256MiB 栈，仍需核对目标评测栈限制。

```
template <class T = long long> struct CentroidSum
{
    struct Record
    {
        int centroid, distance, branch;
    };
    int n;
    vector<vector<int>> g;
    vector<int> siz, removed;
    vector<T> value;
    vector<vector<Record>> path;
    vector<Fenwick<T>> all;
    vector<vector<Fenwick<T>>> part;

    CentroidSum(int n)
        : n(n), g(n), siz(n), removed(n), value(n), path(n), all(n), part(n)
    {
        assert(n > 0);
    }

    void add(int u, int v)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

子树大小、重心与距离记录（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (int v : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (int v : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, int d, int c, int b, vector<T> &bucket)
    {
        path[u].push_back({c, d, b});
        if ((int)bucket.size() <= d + 1) bucket.resize(d + 2);
        bucket[d + 1] += value[u];
        for (int v : g[u])
            if (v != p && !removed[v]) collect(v, u, d + 1, c, b, bucket);
    }
```

线性构建距离桶和递归分解（接上页同一结构体）：

```
    // Raw distance d is stored at 1-based Fenwick index d+1.
    void make_bit(Fenwick<T> &bit, vector<T> bucket)
    {
        bit.n = (int)bucket.size() - 1;
        bit.a = move(bucket);
        for (int i = 1; i <= bit.n; i++)
            if (i + (i & -i) <= bit.n) bit.a[i + (i & -i)] += bit.a[i];
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, 0, -1});
        vector<T> total{T{}, value[c]};
        for (int v : g[c])
        {
            if (removed[v]) continue;
            vector<T> bucket(1);
            collect(v, c, 1, c, (int)part[c].size(), bucket);
            if (total.size() < bucket.size()) total.resize(bucket.size());
            for (int d = 1; d < (int)bucket.size(); d++) total[d] += bucket[d];
            part[c].emplace_back();
            make_bit(part[c].back(), move(bucket));
        }
        make_bit(all[c], move(total));
        for (int v : g[c])
            if (!removed[v]) decompose(v);
    }
```

重建、点权赋值与距离球求和（接上页同一结构体）：

```
    // Rebuild resets all values and indices; do not change the tree after building.
    void build(const vector<T> &values)
    {
        assert((int)values.size() == n);
        value = values;
        fill(removed.begin(), removed.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            part[u].clear();
        }
        decompose(0);
    }

    void set(int u, T x)
    {
        assert(0 <= u && u < n);
        T delta = x - value[u];
        value[u] = x;
        for (auto [c, d, b] : path[u])
        {
            all[c].add(d + 1, delta);
            if (b != -1) part[c][b].add(d + 1, delta);
        }
    }

    T prefix(const Fenwick<T> &bit, long long radius) const
    {
        if (radius < 0) return T{};
        return bit.sum(radius >= bit.n ? bit.n : (int)radius + 1);
    }

    T query(int u, long long radius) const
    {
        assert(0 <= u && u < n);
        T answer{};
        if (radius < 0) return answer;
        for (auto [c, d, b] : path[u])
        {
            answer += prefix(all[c], radius - d);
            if (b != -1) answer -= prefix(part[c][b], radius - d);
        }
        return answer;
    }
};
```

模板题使用：Luogu P6329 【模板】点分树 / 震波

题意：固定单位边树上修改点权，查询到中心距离不超过 k 的权值和。操作 0 查询，操作 1 赋值；每次先将 x 及 y/k 与最近一次查询答案异或，再把 1-based 顶点 x 减一。更新不改变 last，操作类型不异或。n,m≤100000，点权/新值在 1..10000，解码后 k 在 0..n-1。构造后先 build；原树递归 DFS 需要足够栈空间。

所需模板：Fenwick（第 215 页，E.1 节）、CentroidSum（第 33 页，2.7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6329

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    std::vector<long long> value(n);
    for (auto &x : value) std::cin >> x;
    CentroidSum tree(n);
    for (int i = 1; i < n; i++)
    {
        int u, v;
        std::cin >> u >> v;
        tree.add(u - 1, v - 1);
    }
    tree.build(value);
    long long last = 0;
    while (m--)
    {
        int op;
        long long x, y;
        std::cin >> op >> x >> y;
        x ^= last;
        y ^= last;
        if (op == 0)
        {
            last = tree.query((int)x - 1, y);
            std::cout << last << '\n';
        }
        else
            tree.set((int)x - 1, y);
    }
}
```

2.7.2 动态点分树：有符号边权活动点直径与端点

问题描述：固定的一棵树，边权可正可负；维护一个“活动点”集合，支持单点激活/取消，并随时查询活动点之间的最大路径权值（允许两端为同一点，此时为 0）及一对取到最大值的端点。

核心思路：建点分树。对重心 c，它自身和删去 c 后的每个分支各对应一个桶 `bag`，桶中存该部分活动点到 c 的距离；`path[u]` 记录 u 在每层重心下的（重心、桶号、距离）。`top[c]` 保存 c 的每个非空桶的最大值，取不同桶的前两名相加，就是经过 c 且两端位于不同分支的最长真实路径（`candidate`），所有重心的候选放在全局集合 `best` 中。修改点 u 时沿 `path[u]` 逐层：先删掉旧候选和旧桶顶，更新桶，再放回新桶顶和新候选。查询取 `best` 最大值，若不为正则返回任一活动点自身、距离 0。

思路中的情况	代码中的处理	行
重心自身的桶	重心 c 以距离 0 放入自己的桶	74
每个分支一个桶	DFS 记下分支内各点到 c 的距离与桶号	81
修改前撤销	从 `best` 删去 c 的旧候选，从 `top[c]` 删去旧桶顶	123
更新桶并恢复	插入或删除 (d, u)，再放回新桶顶和新候选	129
重心候选	`top[c]` 中来自不同桶的前两名距离相加	111
查询	最大候选不为正时，取单个活动点，答案 0	151

接口契约： CentroidDiameter(n)，固定连通无向树、0-based 点号，n>0 且 2n 不溢出 int。add(u,v,w) 仅在首次 build 前；允许负边权，所有简单路径权值和必须能用 long long 表示。build() 递归分解并清空活动集，可反复重建；set(u,on) 幂等启停点。query() 返回 optional<tuple<long long,int,int»，内容为最大距离与任意一对达到最优值的活动端点，允许两端相同，所以非空集答案至少 0；空集 nullopt。各重心按不同分支取最大距离，只合并跨分支真实路径，不能套用非负权直径端点摘要；并列不承诺最小编号。空集预处理 O(n log n)、set O($\log^2$ n)、query O(1)，空间 O(n log n)；逐点激活全树 O(n $\log^2$ n)。支持值拷贝；active 只读用于反色，其余字段与 sum/size_dfs/centroid/collect/decompose/candidate 为内部状态。DFS 保持递归，树输入合法性与足够栈空间由调用方保证。

```
struct CentroidDiameter
{
    using ll = long long;
    using Item = tuple<ll, int, int>;
    using Best = tuple<ll, int, int, int>;
    struct Entry
    {
        int c, b;
        ll d;
    };
    int n;
    bool built = false;
    vector<vector<pair<int, ll>>> g;
    vector<vector<Entry>> path;
    vector<int> siz, removed, active;
    vector<std::set<pair<ll, int>>> bag;
    vector<std::set<Item>> top;
    std::set<Best> best;
    std::set<int> live;

    CentroidDiameter(int n)
        : n(n), g(n), path(n), siz(n), removed(n), active(n), top(n)
    {
        assert(0 < n && n <= INT_MAX / 2);
    }

    ll sum(ll a, ll b) const
    {
        assert(b >= 0 ? a <= LLONG_MAX - b : a >= LLONG_MIN - b);
        return a + b;
    }

    void add(int u, int v, ll w = 1)
    {
        assert(!built && 0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
```

递归分解与不同分支的距离（接上页同一结构体）：

```
    void size_dfs(int u, int p)
    {
        siz[u] = 1;
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            size_dfs(v, u);
            siz[u] += siz[v];
        }
    }

    int centroid(int u, int p, int total)
    {
        for (auto [v, w] : g[u])
            if (v != p && !removed[v] && siz[v] > total / 2)
                return centroid(v, u, total);
        return u;
    }

    void collect(int u, int p, ll d, int c, int b)
    {
        path[u].push_back({c, b, d});
        for (auto [v, w] : g[u])
        {
            if (v == p || removed[v]) continue;
            collect(v, u, sum(d, w), c, b);
        }
    }

    void decompose(int entry)
    {
        size_dfs(entry, -1);
        int c = centroid(entry, -1, siz[entry]);
        removed[c] = 1;
        path[c].push_back({c, (int)bag.size(), 0});
        bag.emplace_back();
        for (auto [v, w] : g[c])
        {
            if (removed[v]) continue;
            int b = bag.size();
            bag.emplace_back();
            collect(v, c, w, c, b);
        }
        for (auto [v, w] : g[c])
            if (!removed[v]) decompose(v);
    }
```

清空重建与重心候选（接上页同一结构体）：

```
    // Fixed tree, signed weights. Rebuild clears all active points.
    void build()
    {
        fill(removed.begin(), removed.end(), 0);
        fill(active.begin(), active.end(), 0);
        for (int u = 0; u < n; u++)
        {
            path[u].clear();
            top[u].clear();
        }
        bag.clear();
        best.clear();
        live.clear();
        decompose(0);
        built = true;
    }

    optional<Best> candidate(int c) const
    {
        if (top[c].size() < 2) return nullopt;
        auto it = top[c].rbegin();
        auto [x, u, a] = *it++;
        auto [y, v, b] = *it;
        if (u > v) swap(u, v);
        return Best{sum(x, y), u, v, c};
    }
```

活动点设置与端点查询（接上页同一结构体）：

```
    void set(int u, bool on)
    {
        assert(built && 0 <= u && u < n);
        if (active[u] == on) return;
        active[u] = on;
        if (on) live.insert(u);
        else live.erase(u);
        for (auto [c, b, d] : path[u])
        {
            if (auto old = candidate(c)) best.erase(*old);
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].erase({x, v, b});
            }
            if (on) bag[b].insert({d, u});
            else bag[b].erase({d, u});
            if (!bag[b].empty())
            {
                auto [x, v] = *bag[b].rbegin();
                top[c].insert({x, v, b});
            }
            if (auto now = candidate(c)) best.insert(*now);
        }
    }

    // Maximum distance and any attaining pair; equal endpoints are allowed.
    optional<Item> query() const
    {
        assert(built);
        if (live.empty()) return nullopt;
        if (!best.empty())
        {
            auto [d, u, v, c] = *best.rbegin();
            if (d > 0) return Item{d, u, v};
        }
        int u = *live.begin();
        return Item{0, u, u};
    }
};
```

2.8 树哈希

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

2.8.1 确定性有根与无根树同构

问题描述：判断树同构：为有根树或无根树计算一个编号，两棵树同构当且仅当编号相同（同一对象内比较）；无根树返回一对编号。

核心思路：AHU 方法：有根子树的编号由“儿子编号的有序列表”唯一确定。DFS 求出所有儿子的编号后排序，在 `codes` 这个 `map` 中查找该列表，没出现过就分配新编号，因此编号相等等价于同构，没有哈希碰撞。无根

树先一次 DFS 找重心（最大子树不超过 $n/2$ 的点，至多两个）：只有一个重心时以它为根编码；有两个时分别以两者为根编码，排序后成对返回。

接口契约： TreeIsomorphism iso；rooted(g,root) 返回有根树编号，unrooted(g) 返回无根树 pair 编号。g 是 0-based 非空简单无向连通树，邻接表双向各存一次；root 在 0..n-1。递归 DFS，不验证非法图。所有待比较的树必须共用同一个 iso；编号依赖插入次序，不同对象或清空 codes 前后的编号不能直接比较。对子节点编号排序并用 map 精确驻留，无随机哈希碰撞。无根树取重心：单重心返回 (code,0)，双重心分别以两点为根编码并排序。支持单点；不考虑点/边标签，也不返回顶点映射。codes 只读，编号数须 <INT_MAX，n<=INT_MAX。S 为本次结束时驻留签名数，一次 n 点调用 O(n log(n+1)+n log(S+1))，临时 O(n)，跨调用累计存储至多 O(T)，T 为累计编码遍历的顶点数。

```
struct TreeIsomorphism
{
    map<vector<int>, int> codes;

    int rooted(const vector<vector<int>> &g, int root)
    {
        assert(0 <= root && root < int(g.size()));
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            vector<int> children;
            for (int v : g[u])
            {
                if (v == p) continue;
                children.push_back(self(self, v, u));
            }
            sort(children.begin(), children.end());
            int next = int(codes.size()) + 1;
            return codes.emplace(children, next).first->second;
        };
        return dfs(dfs, root, -1);
    }
```

继续求解（接上页同一结构体）：

```
    pair<int, int> unrooted(const vector<vector<int>> &g)
    {
        int n = g.size();
        assert(n > 0);
        vector<int> centers;
        auto dfs = [&](auto &&self, int u, int p) -> int
        {
            int size = 1, largest = 0;
            for (int v : g[u])
            {
                if (v == p) continue;
                int child = self(self, v, u);
                size += child;
                largest = max(largest, child);
            }
            largest = max(largest, n - size);
            if (largest <= n / 2) centers.push_back(u);
            return size;
        };
        dfs(dfs, 0, -1);
        int a = rooted(g, centers[0]);
        if (centers.size() == 1) return {a, 0};
        int b = rooted(g, centers[1]);
        if (a > b) swap(a, b);
        return {a, b};
    }
};
```

模板题使用：P5043 【模板】树同构 / [BJOI2015] 树的同构

题意：M 棵无根树，每棵 N 个点，1<=N,M<=50。输入 N 个父亲编号，0 表示根；转换无向树后忽略输入根。输出与本树同构的最早树编号；所有树共用同一驻留表。

所需模板：TreeIsomorphism（第 43 页，2.8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5043

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    TreeIsomorphism iso;
    map<pair<int, int>, int> first;
    for (int id = 1; id <= m; id++)
    {
        int n;
        cin >> n;
        vector<vector<int>> g(n);
        for (int u = 0; u < n; u++)
        {
            int p;
            cin >> p;
            if (p == 0) continue;
            p--;
            g[u].push_back(p);
            g[p].push_back(u);
        }
        auto code = iso.unrooted(g);
        auto result = first.emplace(code, id);
        cout << result.first->second << '\n';
    }
}
```

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

2.8.2 双 64 位随机树哈希（Mix13）

功能：对一棵有根树，求每个点子树的 128 位（两个 64 位）随机哈希值；两棵子树同构时哈希一定相等，不同构时以极高概率不等，可用于统计不同构子树的种类或比较多棵树。

核心思路：哈希定义为 $h(u) = s + \sum_{v\in \mathrm{son}(u)} \mathrm{mix13}(h(v))$，模 2^{64} 自然溢出，两个分量用不同种子 s_1, s_2 分别计算；求和与儿子顺序无关，因此不需要排序。`mix13` 是 64 位整数混淆函数，使不同子树的贡献难以相互抵消。代码先从根 BFS 得到父亲 `fa` 和层序 `q`，再逆层序把每个点的 `mix13` 值加到父亲上，非递归。

接口契约：tree_hash(g,root,s1,s2) 对 0-based 树 g（非空、连通、简单无向）以 root 为根，返回每个点 u 的有根子树哈希对 h[u]：h[u]=(s1,s2)+ 各儿子 v 的 (mix13(h[v].first),mix13(h[v].second)) 之和，全部模 2^64 自然溢出；叶子为 (s1,s2)。儿子顺序无关、不需要排序。种子为非零且互异的随机 64 位数，由调用方生成（如 mt19937_64 以时间为种子），需要互相比较的所有树必须共用同一对种子。h[u] 相等当且仅当两子树同构（概率意义，128 位哈希，非确定性证明）；统计不同子树种类时对 h 排序去重。BFS 序非递归，O(n) 时间、O(n) 空间，百万点长链不会爆栈。需要确定性结论时用 TreeIsomorphism。

```cpp
// Double 64-bit random rooted-tree hash (Mix13); h[u] equal <=> subtrees isomorphic w.h.p.
// 0-based tree, any root. Trees compared together must share the same nonzero seeds s1 != s2.
unsigned long long mix13(unsigned long long x)
{
    x ^= x >> 30, x *= 0xbf58476d1ce4e5b9ULL;
    x ^= x >> 27, x *= 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

vector<pair<unsigned long long, unsigned long long>>
tree_hash(const vector<vector<int>> &g, int root, unsigned long long s1, unsigned long long s2)
{
    int n = g.size();
    vector<int> fa(n, -1), q{root};
    for (int i = 0; i < (int)q.size(); i++)
        for (int v : g[q[i]])
            if (v != fa[q[i]]) fa[v] = q[i], q.push_back(v);
    vector<pair<unsigned long long, unsigned long long>> h(n, {s1, s2});
    for (int i = n - 1; i > 0; i--)
    {
        int u = q[i];
        h[fa[u]].first += mix13(h[u].first);
        h[fa[u]].second += mix13(h[u].second);
    }
    return h;
}
```

模板题使用：UOJ #763 树哈希

题意：n≤10^6 个点的树以 1 为根，求互不同构的有根子树种类数。点号减一转为 0-based，以时间为种子生成两枚 64 位种子，对所有 h[u] 排序去重计数。

所需模板：tree_hash（第 47 页，2.8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://uoj.ac/problem/763

```
int main()
{
    int n;
    scanf("%d", &n);
    vector<vector<int>> g(n);
    for (int i = 1; i < n; i++)
    {
        int a, b;
        scanf("%d%d", &a, &b);
        a--, b--;
        g[a].push_back(b), g[b].push_back(a);
    }
    mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
    unsigned long long s1 = rng() | 1, s2 = rng() | 1;
    auto h = tree_hash(g, 0, s1, s2);
    sort(h.begin(), h.end());
    printf("%d\n", int(unique(h.begin(), h.end()) - h.begin()));
}
```

相关应用：Deterministic sorted-child interning and centroid rooting; exact alternative to randomized tree hashing.

2.8.3 树哈希 O(n) 换根

功能：在 `tree_hash` 的基础上，求以每个点 u 为根时整棵树的哈希 $f(u)$，用于无根树比较（取所有 $f(u)$ 的最小值等作代表）。

核心思路：换根 DP。根处 $f(\text{root}) = h(\text{root})$。按 BFS 序处理点 u（父亲 p 已算好）：以 p 为根的整树哈希 $f(p)$ 去掉儿子 u 的贡献 $\text{mix13}(h(u))$，就是“以 u 为根时 p 那一侧子树”的哈希；它作为 u 的新儿子，所以 $f(u) = h(u) + \text{mix13}(f(p) - \text{mix13}(h(u)))$，两个分量分别计算。

接口契约：tree_hash_reroot(g,root,h) 由同一根的 h=tree_hash(g,root,s1,s2) 求 f[u]：以 u 为根时整棵树的哈希。父方向子树的哈希为 f[p]-mix13(h[u])，故 f[u]=h[u]+mix13(f[p]-mix13(h[u]))，两分量分别计算。无根树比较取所有 f[u] 的最小值（或按重心）作为整树代表。O(n) 时间与空间，非递归。

```
// f[u] = hash of the whole tree rooted at u, from h = tree_hash(g, root, s1, s2). O(n).
vector<pair<unsigned long long, unsigned long long>>
tree_hash_reroot(const vector<vector<int>> &g, int root, const vector<pair<unsigned long long, unsigned long long>> &h)
{
    int n = g.size();
    vector<int> fa(n, -1), q{root};
    for (int i = 0; i < (int)q.size(); i++)
        for (int v : g[q[i]])
            if (v != fa[q[i]]) fa[v] = q[i], q.push_back(v);
    auto f = h;
    for (int i = 1; i < n; i++)
    {
        int u = q[i], p = fa[u];
        // the parent side seen from u is f[p] without u's own contribution
        f[u].first += mix13(f[p].first - mix13(h[u].first));
        f[u].second += mix13(f[p].second - mix13(h[u].second));
    }
    return f;
}
```

第三章　最短路问题

3.1　最短路

3.1.1　非负权最短路

问题描述：给定非负边权有向图和源点 s，求 s 到所有点的最短距离，并可沿前驱 `pre` 恢复到某点的最短路径。

核心思路：小根堆按距离取点，懒删除：弹出的 (d,u) 若 d 不等于当前 `dis[u]` 说明已过期，直接跳过；否则 u 的距离已确定，用它松弛所有出边，更新 `dis` 和 `pre` 并把新距离入堆。`path(t)` 从 t 沿 `pre` 走回 s 后反转。

接口契约：add(u,v,w)、run(s)，dis 为距离，pre 为前驱，path(t) 返回点序列。不可达用 LLONG_MAX；要求所有有限最短距离严格小于该哨兵，w≥0。懒删除堆在多重边图中按 O(n+m log(m+1)) 计，空间 O(n+m)；简单图常写 O((n+m)log(n+1))。最短路树：run 后每个 v≠s 的 pre 入边构成一棵最短路树；要总边权最小，就在满足 dis[u]+w=dis[v] 的入边中选 w 最小者。Johnson 全源最短路：加超级源以 0 权连所有点，用 BellmanFord 求势 h，把边权改成 w+h[u]-h[v]≥0 后对每个起点跑 Dijkstra，真实距离为 d+h[t]-h[s]，O(nm log m)。

```
struct Dijkstra
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;
    vector<ll> dis;
    vector<int> pre;

    Dijkstra(int n) : n(n), g(n + 1) {}

    void add(int u, int v, ll w)
    {
        assert(w >= 0);
        g[u].push_back({v, w});
    }

    void run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        dis[s] = 0;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d != dis[u]) continue;
            for (auto [v, w] : g[u])
                if (w < inf - d && d + w < dis[v])
                {
                    dis[v] = d + w;
                    pre[v] = u;
                    q.push({dis[v], v});
                }
```

```
        }
    }

    vector<int> path(int t) const
    {
        if (dis[t] == inf) return {};
        vector<int> ans;
        for (; t != -1; t = pre[t]) ans.push_back(t);
        reverse(ans.begin(), ans.end());
        return ans;
    }
};
```

模板题使用：Luogu P4779

题意：给非负权有向图和源点 s，输出 s 到各点的最短距离。加完边后 run(s)，读取 dis[1..n]；本题保证源点可达所有点，一般输入的不可达值须按目标题目格式另行转换。

所需模板：Dijkstra（第 50 页，3.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4779

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m, s;
    std::cin >> n >> m >> s;
    Dijkstra graph(n);
    while (m--)
    {
        int u, v;
        long long w;
        std::cin >> u >> v >> w;
        graph.add(u, v, w);
    }
    graph.run(s);
    for (int i = 1; i <= n; i++) std::cout << graph.dis[i] << (i == n ? '\n' : ' ');
}
```

3.1.2 Bellman–Ford：负环影响与路径方案

问题描述：给定可含负权的有向图和起点 s，求 s 到各点的最短距离及一条最短路的原边序列；若某点可经由从 s 可达的负环到达，则其距离为 $-\infty$，并给出一个这样的负环。$s=0$ 时视为从所有点同时出发，用于判全图负环。

核心思路：做至多 n 轮“扫描全部边做松弛”，`pre[v]` 记最后一次更新 v 的边。无负环时 $n-1$ 轮内收敛；若第 n 轮仍有松弛，被更新的点必受负环影响，标记 `neg`。从第 n 轮最后被松弛的点沿 `pre` 回退 n 步必落在环上，再绕一圈收集环边。最后把 `neg` 沿边反复传播到不动点，凡能从负环到达的点都标为 $-\infty$。路径方案沿 `pre` 倒推后翻转。

思路中的情况	代码中的处理	行
最多 n 轮全边松弛	某轮无更新即提前结束；`last` 记本轮最后被松弛的点	31
第 n 轮仍可松弛	该点距离为 $-\infty$，置 `neg`	39
进入负环	从 `last` 沿 `pre` 回退 n 步	46
收集负环	从环上点 x 绕回自身，记录边号后翻转为正向	47
传播 $-\infty$	沿边扩散 `neg` 直到不再变化	54

接口契约： BellmanFord(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 加有向边并返回 0-based 原边号，允许重边、自环和完整 signed64 边权，总边数 ≤INT_MAX。run(s) 返回是否不存在从 s 可达的负环；run(0) 把全体点初值设 0，等价于超级源点，用于查全图负环。每次 run 重置状态；add 后旧结果失效，须重新 run。

运行后 dis[v]==inf 表示不可达；neg[v] 为真表示从起点可达的负环能到达 v，距离为负无穷（这时不可读取 dis 作为答案）；其他点的 dis 为有限 int128 最短距离。path(v) 返回有限最短路的正向原边号；不可达、负无穷或空路径都返回空 vector，须结合 dis/neg 判定。cycle 为空表示无相关负环，否则是一个相关负环的正向原边号，可通过 e[id].u/v/w 还原顶点和权值。run(0) 的有限距离是从任意起点出发的最小值，允许零边路径，不是某个固定单源距离。

O(nm+n+m) 时间、O(n+m) 空间；path 输出 O(路径长)。inf=2^126，全过程 int128 累加，不能无条件转为 long long。在给定 n,m 的 int 边界内，n 轮最多 nm 次更新，标签对应游走至多 nm 条边，绝对值 <2^125，不碰 inf 也不溢出。e/g/dis/pre/neg/cycle 和 ready 只读，不手改。

```
struct BellmanFord
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<long long> dis;
    vector<int> pre, neg, cycle; // neg[v]: distance is -inf; cycle: edge ids of one negative cycle

    BellmanFord(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts every vertex at 0 to find any negative cycle. True iff none found.
    bool run(int s)
    {
        dis.assign(n + 1, s ? inf : 0);
        pre.assign(n + 1, -1);
        neg.assign(n + 1, 0);
        cycle.clear();
        dis[s] = 0;
        int last = -1;
        for (int step = 0; step < n; step++)
        {
            last = -1;
            for (int id = 0; id < (int)e.size(); id++)
            {
```

```
                auto [u, v, w] = e[id];
                if (dis[u] == inf || dis[u] + w >= dis[v]) continue;
                dis[v] = dis[u] + w, pre[v] = id, last = v;
                if (step == n - 1) neg[v] = 1;
            }
            if (last == -1) break;
        }
        if (last != -1)
        {
            int x = last;
            for (int i = 0; i < n; i++) x = e[pre[x]].u;
            for (int y = x; cycle.empty() || y != x; y = e[pre[y]].u) cycle.push_back(pre[y]);
            reverse(cycle.begin(), cycle.end());
        }
        for (bool changed = true; changed;) // everything reachable from round n is -inf
        {
            changed = false;
            for (auto [u, v, w] : e)
                if (neg[u] && !neg[v]) neg[v] = 1, changed = true;
        }
        return cycle.empty();
    }

    // Finite shortest path as edge ids; empty for unreachable, -inf or t=s.
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf || neg[t]) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_B / 含负边的单源最短路

题意：0-based 有向图，n≤1000、m≤2000、边权绝对值 ≤10000。从 r 可达负环时输出 NEGATIVE CYCLE，否则逐点输出距离，不可达输出 INF。转成模板 1-based 编号；本题有限距离在 signed64 内，故可显式转 long long 输出。不可达位置的负环不影响此题判断。

所需模板：BellmanFord（第 51 页，3.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_B

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, s;
    cin >> n >> m >> s;
    BellmanFord t(n);
    while (m--)
    {
        int u, v, w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run(s + 1)) cout << "NEGATIVE CYCLE\n";
    else
        for (int u = 1; u <= n; u++)
        {
            if (t.dis[u] == BellmanFord::inf) cout << "INF\n";
            else cout << (long long)t.dis[u] << '\n';
        }
}
```

3.1.3 SPFA：队列松弛、负环判定与有限路径

问题描述：给定可含负权的有向图和起点 s，判断是否存在从 s 可达的负环；若不存在，求 s 到各点的最短距离与一条最短路的原边序列。$s=0$ 时所有点初始距离为 0，可判全图负环，也可直接求差分约束的一组可行解。

核心思路：队列优化的 Bellman–Ford：只有距离被改小的点才入队，再用它松弛出边；`in` 标记点是否已在队中以免重复入队。每次严格松弛时令 `len[v]=len[u]+1`，表示当前标签对应行走的边数；无负环时最短路是简单路径，边数不超过 $n-1$，因此一旦 `len[v]` 达到 n 就立即报告负环。`pre[v]` 记最后更新 v 的边，用于倒推路径。**接口契约：**Spfa(n)，点号 1..n，0<n<INT_MAX。add(u,v,w) 返回 0-based 原边号，允许重边、自环、完整 signed64 权值，边数 ≤INT_MAX；加边令 valid=false。run(s) 查从 s 可达的负环，s=0 同时初始化所有点为 0 以查全图。返回 false 时 valid=false，全部 dis/pre 均为未完成状态，不能读取为最短路或恢复路径；需要负环方案或逐点负无穷分类用 BellmanFord。

run 返回 true 时 valid=true，dis[v]==inf 为不可达，其余为 int128 最短距离；run(0) 距离为任意起点出发的最小值，且本身就是全部约束 x_v−x_u≤w 的一个可行解。path(v) 返回正向原边号；不可达与零边路径均为空，结合 dis 区分。每次 run 重置临时队列/标记/路径边数，支持反复改起点与对象复制。

FIFO 队列，最坏 O(nm+n+m) 时间，O(n+m) 空间，不保证近线性；队列去重不等于永久访问标记。严格松弛后维护标签所代表游走的边数，达到 n 立即判负环。停止前标签最多 n 条边，绝对值 ≤n · 2^63<2^94，int128 与 inf=2^126 足够。不要无条件转 long long；e/g/dis/pre/valid 只读，其余按接口使用。

```
struct Spfa
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
```

```
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<int>> g;
    vector<long long> dis;
    vector<int> pre;

    Spfa(int n) : n(n), g(n + 1) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        g[u].push_back(int(e.size()) - 1);
        return int(e.size()) - 1;
    }

    // 1-based; s=0 starts from every vertex (difference constraints). False on a negative cycle.
    bool run(int s)
    {
        dis.assign(n + 1, inf);
        pre.assign(n + 1, -1);
        vector<int> in(n + 1), len(n + 1);
        queue<int> q;
        for (int u = 1; u <= n; u++)
            if (!s || u == s) dis[u] = 0, in[u] = 1, q.push(u);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            in[u] = 0;
            for (int id : g[u])
            {
                int v = e[id].v;
                if (dis[u] + e[id].w >= dis[v]) continue;
                dis[v] = dis[u] + e[id].w, pre[v] = id, len[v] = len[u] + 1;
                if (len[v] >= n) return false;
                if (!in[v]) in[v] = 1, q.push(v);
            }
        }
        return true;
    }

    // Edge ids of the shortest path to t; empty if unreachable or t is a start.
    vector<int> path(int t) const
    {
        vector<int> ids;
        if (dis[t] == inf) return ids;
        for (; pre[t] != -1; t = e[pre[t]].u) ids.push_back(pre[t]);
        reverse(ids.begin(), ids.end());
        return ids;
    }
};
```

模板题使用：Luogu P3385 / 从 1 可达的负环

题意：T 组图，每条输入边权非负时加双向边，负数仅加 u 到 v；run(1) 只查从 1 可达的负环，有则 YES，否则 NO。m 是输入记录数，不是最终边数；不可达部分的负环不影响本题。

所需模板：Spfa（第 54 页，3.1.3 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3385

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while (T--)
    {
        int n, m;
        cin >> n >> m;
        Spfa t(n);
        while (m--)
        {
            int u, v, w;
            cin >> u >> v >> w;
            t.add(u, v, w);
            if (w >= 0) t.add(v, u, w);
        }
        cout << (t.run(1) ? "NO" : "YES") << '\n';
    }
}
```

3.1.4 Floyd 全源最短路与原边方案

问题描述：给定可含负权、重边的有向图，求所有点对之间的最短距离，并能输出任一点对的一条最短路（原边序列）；若图中有负环则报告失败。

核心思路：`dis[u][v]` 初值取 $u \to v$ 重边中的最小权，`nxt[u][v]` 记最短路的第一条原边。按 $k = 1..n$ 逐个允许 k 作中转点，用 $\mathrm{dis}[u][k] + \mathrm{dis}[k][v]$ 更新 $\mathrm{dis}[u][v]$，更新时第一条边继承 `nxt[u][k]`；只有两段都可达才相加。处理 k 前若 $\mathrm{dis}[k][k] < 0$ 说明存在经过 k 的负环，立即返回。输出路径时从 s 反复走 `nxt[s][t]` 这条边，直到到达 t。

接口契约：Floyd(n)，点号 1..n，0<n<INT_MAX；add(u,v,w) 返回 0-based 原边号，有向边，允许重边、自环及完整 signed64 权值，边数 ≤INT_MAX。无向图需分别加入双向边。run() 重新建立矩阵；存在任何负环返回 false，valid=false，全部结果不可读，不能只取未受负环影响的点对。

成功时 dis[u][v]==inf 为不可达，其他为 int128 最短距离，dis[u][u]=0。path(u,v) 返回正向原边号，并列任选；不可达和 u=v 均为空，结合 dis 区分。nxt 为恢复路径的首条原边号矩阵，通常只需调用 path；e/dis/nxt/valid 只读。add 清除 valid，支持重复运行、加边后重跑与独立值拷贝。

run 时间 $O(n^3+m)$，空间 $O(n^2+m)$，path 时间与输出空间 O(路径长)。每次处理 k 之前检查 dis[k][k]<0，立即拒绝，不能延后到所有松弛结束，否则负环下中间量可指数增长。通过的枢轴不含负环；有限标签取简单路径或闭合路径，拼接候选绝对值 $<2n \cdot 2\hat{}63<2\hat{}95$，int128 与 inf=2^126 足够。两段都有限才相加；不要无条件转 long long。矩阵占用约 $20(n+1)^2$ 字节，另计 vector 和原边表，n 应按时间/内存选择。

```
struct Floyd
{
    static constexpr long long inf = LLONG_MAX / 4;
    struct Edge
```

```
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<vector<long long>> dis;
    vector<vector<int>> nxt;

    Floyd(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // 1-based. False on a negative cycle; then every result is invalid.
    bool run()
    {
        dis.assign(n + 1, vector<long long>(n + 1, inf));
        nxt.assign(n + 1, vector<int>(n + 1, -1));
        for (int u = 1; u <= n; u++) dis[u][u] = 0;
        for (int id = 0; id < (int)e.size(); id++)
        {
            auto [u, v, w] = e[id];
            if (w < dis[u][v]) dis[u][v] = w, nxt[u][v] = id;
        }
        for (int k = 1; k <= n; k++)
        {
            if (dis[k][k] < 0) return false;
            for (int u = 1; u <= n; u++)
                if (dis[u][k] != inf)
                    for (int v = 1; v <= n; v++)
                        if (dis[k][v] != inf && dis[u][k] + dis[k][v] < dis[u][v])
                            dis[u][v] = dis[u][k] + dis[k][v], nxt[u][v] = nxt[u][k];
        }
        return true;
    }

    // Edge ids from s to t; empty if unreachable or s=t.
    vector<int> path(int s, int t) const
    {
        vector<int> ids;
        if (dis[s][t] == inf) return ids;
        for (; s != t; s = e[ids.back()].v) ids.push_back(nxt[s][t]);
        return ids;
    }
};
```

模板题使用：AOJ GRL_1_C / All Pairs Shortest Path

题意：0-based 有向图输入转换为 1-based；整图有负环输出 NEGATIVE CYCLE，否则逐行输出距离矩阵，不可达为 INF。n≤100、m≤9900、|w|≤20000000，有限结果可转 long long。题面无自环重边，模板额外支持。

所需模板：Floyd（第 56 页，3.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=GRL_1_C&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Floyd t(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        t.add(u + 1, v + 1, w);
    }
    if (!t.run())
    {
        cout << "NEGATIVE CYCLE\n";
        return 0;
    }
    for (int u = 1; u <= n; u++)
    {
        for (int v = 1; v <= n; v++)
        {
            if (v > 1) cout << ' ';
            if (t.dis[u][v] == Floyd::inf) cout << "INF";
            else cout << (long long)t.dis[u][v];
        }
        cout << '\n';
    }
}
```

3.1.5 杭州 2023 G：开放时间的单位边最短路

QOJ8236。初始蛇身从头到尾编号 $0,\dots,k-1$，第 $i\ge 1$ 节所在点开放时间为 $k-i$（尾格为 1），其余为零；该限制必要且充分。转移 $d_v=\max(d_u+1,\mathrm{open}_v)$。按距离递增处理时首次发现即最优：候选等于 d_u+1 的入 FIFO，其余入开放时间桶，每次取两者较小者；桶指针只扫到下一个 FIFO 距离。$R=\max \mathrm{open}$，时间 $O(V+E+R)$，空间 $O(V+R)$。仅适用于所有边权为 1；一般非负权用优先队列。答案用 unsigned long long 累加距离平方（模 2^{64}）。

3.2 k 短路

3.2.1 非负权严格次短行走

问题描述：给定非负权有向图和起点 s，对每个点 v 求 s 到 v 的最短行走长度，以及严格大于它的最小行走长度（严格次短路）；行走可重复经过点和边。

核心思路：Dijkstra 的推广：每个点维护两个标签 `dis[v][0]`（最短）和 `dis[v][1]`（严格次短），堆中可同时有同一点的两种长度。弹出 (d,u) 后若 d 已大于 `dis[u][1]` 就丢弃，否则用 $d+w$ 松弛 v：若比最短还小，就与 `dis[v][0]` 交换，被替换下来的旧最短值接着作为次短候选；候选值严格介于最短和次短之间时更新次短。每次更新都把新值入堆。

接口契约：StrictSecondShortest(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，无向边添加两次。允许重边、自环及零权，0<=w<LLONG_MAX。run(s) 返回 dis[v][0/1]，分别为最短距离与严格大于它的最小距离；同长路线不重复计数，允许重复点边，s 到自身包含空路。不存在或长度 >=LLONG_MAX 返回 inf=LLONG_MAX；只保证此阈值以下的精确答案，超界扩展跳过。只读 g/n；可重复求解、加边后重算。O((n+m) log(n+m+1)) 时间，O(n+m) 空间，不返回行走方案。

```
struct StrictSecondShortest
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    StrictSecondShortest(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 <= w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<array<ll, 2>> run(int s) const
    {
        assert(1 <= s && s <= n);
        vector<array<ll, 2>> dis(n + 1, {inf, inf});
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        dis[s][0] = 0;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (d > dis[u][1]) continue;
            for (auto [v, w] : g[u])
            {
                if (w >= inf - d) continue;
                ll next = d + w;
                if (next < dis[v][0])
                {
                    swap(next, dis[v][0]);
                    q.push({dis[v][0], v});
                }
                if (dis[v][0] < next && next < dis[v][1])
                {
                    dis[v][1] = next;
                    q.push({next, v});
                }
            }
        }
        return dis;
    }
};
```

3.2.2 正权前 K 短行走

问题描述：给定正权有向图、起点 s、终点 t 和 k，按长度从小到大求 s 到 t 的前 k 短行走的长度；行走可重复经过点和边，不同边序列分别计数。

核心思路：对“到达状态”做 Dijkstra：堆中存 (d,u)，表示一条长度为 d、到达 u 的行走。点 u 第 i 次被弹出时，d 恰为到 u 的第 i 短行走长度，`count[u]` 记录已弹出次数。前 k 次弹出 t 即得答案；每个点最多扩展 k 次，超过的弹出直接丢弃（第 k 短以后的行走不会帮助产生前 k 短答案），已满 k 次的点也不再入堆。

接口契约：KShortestWalks(n)，点号 1..n，n<INT_MAX；add(u,v,w) 添加有向边，要求 0<w<LLONG_MAX，零权边/环不在接口范围内。允许自环与重边，每次 add 视为不同边。run(s,t,k) 按非降序返回至多 k 个行走长度，k>=0；不同边序列分别计数，即使等长。允许重复点边，不是简单路径算法。s=t 时第一个答案为空路 0；若排除空路，调用 k+1 后删除首项，需 k<INT_MAX。终点仍可继续走再回来。长度 >=LLONG_MAX 不返回；只有阈值以下答案有保证。每个顶点至多扩展 k 次，O(n+km log(km+2)) 时间、O(n+km) 空间；k 大时应改用更高效算法。只读 g/n，可重复求解和加边后重算，不返回方案。

```cpp
struct KShortestWalks
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX;
    int n;
    vector<vector<pair<int, ll>>> g;

    KShortestWalks(int n) : n(n), g(n + 1)
    {
        assert(n > 0 && n < INT_MAX);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(0 < w && w < inf);
        g[u].push_back({v, w});
    }
```

继续求解（接上页同一结构体）：

```
    vector<ll> run(int s, int t, int k) const
    {
        assert(1 <= s && s <= n && 1 <= t && t <= n && k >= 0);
        vector<ll> answer;
        if (k == 0) return answer;
        vector<int> count(n + 1);
        priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> q;
        q.push({0, s});
        while (!q.empty())
        {
            auto [d, u] = q.top();
            q.pop();
            if (count[u] == k) continue;
            count[u]++;
            if (u == t)
            {
                answer.push_back(d);
                if (int(answer.size()) == k) break;
            }
            for (auto [v, w] : g[u])
            {
                if (count[v] < k && w < inf - d)
                {
                    q.push({d + w, v});
                }
            }
        }
        return answer;
    }
};
```

第四章　生成树问题

4.1　最小生成树

4.1.1　Kruskal 重构森林与阈值连通块

功能：构建 Kruskal 重构树（森林）：按边权升序（或降序）做 Kruskal，每次成功合并新建一个父节点，权值为该边权。支持查询：只保留权不超过 w（降序时不小于 w，可改为严格）的边时，点 u 所在连通块对应的重构树节点，其子树叶子恰为该连通块。

核心思路：叶子是原点。排序后逐边用并查集合并，`top` 记每个并查集根当前对应的重构树节点；合并时新建节点 x，两个儿子是两侧的 `top`，`val[x]` 为边权。边按顺序处理，所以从叶到根 `val` 单调不减（降序时不增）；新节点编号总大于儿子，可按编号顺序自底向上聚合子树信息。查询时在倍增表 `up` 上从高位到低位跳：祖先的 `val` 满足阈值就跳上去，最终停在满足条件的最高祖先。

代码依赖：dsu（第 213 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：KruskalTree(n)，0≤n≤INT_MAX/2，原点 0..n-1；add(u,v,w) 加入无向边，返回插入编号，边数≤INT_MAX，支持完整 signed64 权、重边、自环、负权和断连。build(descending=false) 重建并返回连通块数；默认升序，true 降序。每次成功合并创建父点：ch 为左右儿子，val 为合并边权；叶点 ch={-1,-1} 且 val 无意义。总点数 k=2n-连通块数，父编号大于子编号；可按 n..k-1 聚合叶点信息，不需要模拟 DFS。up[0] 为父点，根为-1。

component(u,w,strict=false) 返回阈值子图中 u 所属分量的最高重构节点；默认升序保留边权 ≤w，降序保留 ≥w，strict 改成 < 或 >。其子树叶点恰为分量，可从聚合数组查大小、最小值等。同权边必须连续爬完所有合法祖先；不以叶点 val 充当无穷哨兵。构建 O(m log(m+1)+n log(n+1))，空间 O(m+n log(n+1))，查询 O(log(n+1))。依赖 dsu，合并前按大小交换根。支持重复 build、切换方向与值复制；加边后必须重建再查询，构建后成员只读。空图只允许 build、不允许查询。

```
struct KruskalTree
{
    struct Edge
    {
        int u, v;
        long long w;
    };

    int n;
    bool down = false;
    vector<Edge> e;
    vector<long long> val;
    vector<array<int, 2>> ch;
    vector<vector<int>> up;

    KruskalTree(int n) : n(n)
    {
        assert(0 <= n && n <= INT_MAX / 2);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

重建森林与倍增表（接上页同一结构体）：

```
    int build(bool descending = false)
    {
        down = descending;
        ch.assign(n, {-1, -1});
        val.assign(n, 0);
        dsu d(n);
        vector<int> top(n), ids(e.size());
        iota(top.begin(), top.end(), 0);
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(), ids.end(), [&](int a, int b)
        {
            if (e[a].w != e[b].w)
                return down ? e[a].w > e[b].w : e[a].w < e[b].w;
            return a < b;
        });
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            u = d.find(u);
            v = d.find(v);
            if (u == v) continue;
            if (d.sz[u] < d.sz[v]) swap(u, v);
            int x = ch.size();
            ch.push_back({top[u], top[v]});
            val.push_back(w);
            d.merge(u, v);
            top[u] = x;
        }
        int k = ch.size();
        int h = max(1, (int)bit_width((unsigned)k));
        up.assign(h, vector<int>(k, -1));
        for (int u = n; u < k; u++)
        {
            for (int v : ch[u]) up[0][v] = u;
        }
        for (int j = 1; j < h; j++)
        {
            for (int u = 0; u < k; u++)
            {
                int v = up[j - 1][u];
                if (v != -1) up[j][u] = up[j - 1][v];
            }
        }
        return 2 * n - k;
    }
```

阈值分量查询（接上页同一结构体）：

```
    int component(int u, long long w, bool strict = false) const
    {
        assert(0 <= u && u < n && !up.empty());
        for (int j = (int)up.size() - 1; j >= 0; j--)
        {
            int v = up[j][u];
            if (v == -1) continue;
            bool ok = down ? val[v] >= w : val[v] <= w;
            if (strict && val[v] == w) ok = false;
            if (ok) u = v;
        }
        return u;
    }
};
```

为何阈值查询正确 升序处理完所有权不超过 w 的边后，并查集恰好是该阈值子图的连通分量。每次成功合并的两棵子树叶点集合不交，新父点记录其并集；沿叶点到根的内部点权单调不减。因此最高合法祖先的叶点集合就是答案。降序情形反向比较；严格阈值排除等权合并。同权内部点可能连续出现，倍增必须爬到最高合法点。

如何聚合分量 叶点先存原数据，随后按编号 n 到 k-1 合并两个儿子的结果，可求大小、最小值、最大值等。不必为聚合另跑 DFS；这利用的是建点时已经确定的父子拓扑顺序。断连图的各棵树独立，无须添加超级根。

归程建模 先用长度求每个点到 1 号点的最短步行距离。汽车只能走海拔严格大于水位的边，故选降序构建和 strict=true；可在分量内任意点下车，答案为该重构子树的叶点距离最小值。普通最小生成树的总权不能代替这个分量查询。

4.1.2 曼哈顿最小生成树

问题描述：给定平面上 n 个点，两点间边权为曼哈顿距离 $|x_i - x_j| + |y_i - y_j|$，求完全图的最小生成树（总权与边集）。

核心思路：每个点只需与它周围 8 个 45° 区域中各自最近的点连边，MST 必在这些候选边中，对称区域合并后共 4 个方向。`candidates` 每次处理一个方向：按 $x + y$ 升序扫描点 i，`sweep` 以 $-y$ 为键保存还没找到最近点的点；取出满足 $y_j \le y_i$ 且 $x_i - x_j \ge y_i - y_j$ 的点 j（即 i 落在 j 的该区域内），由扫描顺序它就是 j 在该区域里最近的点，连边后删去 j。处理完一个方向后交替做交换 x, y 与取负 x 的坐标变换，四次覆盖所有方向，最后对候选边跑 Kruskal。

代码依赖：dsu（第 213 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Point 为 pair<long long,long long>，坐标支持完整有符号 64 位，点数须 ≤INT_MAX 且内存可分配。candidates(points) 给出最多 4n 条 (weight,u,v) 候选，允许重复边；solve(points) 返回精确 128 位 weight 与原下标边对 edges。空/单点返回 0 且无边，重合点仍为不同顶点；同权最优树不唯一。先转 128 位再变换/加减，输入不变。四方向扫描与 Kruskal 为 O(n log(n+1))、O(n) 空间；调用处选较大 dsu 根在前，不改变普通 dsu 语义。候选保证只针对无约束完整 L1 图，不能直接删禁边后声称仍含受限 MST。

```
struct ManhattanMST
{
    using I = __int128;
    using Point = pair<long long, long long>;
    using Edge = tuple<I, int, int>;
    struct Result
    {
        I weight = 0;
        vector<pair<int, int>> edges;
    };

    static vector<Edge> candidates(const vector<Point> &points)
    {
        assert(points.size() <= INT_MAX);
        int n = int(points.size());
        vector<pair<I, I>> p(points.begin(), points.end());
        vector<int> id(n);
        iota(id.begin(), id.end(), 0);
        vector<Edge> e;
        e.reserve(size_t(n) * 4);
        for (int dir = 0; dir < 4; dir++)
        {
            sort(id.begin(), id.end(), [&](int i, int j)
            {
                return pair{p[i].first + p[i].second, i} <
                       pair{p[j].first + p[j].second, j};
            });
```

活动点扫描与 Kruskal（接上页同一结构体，仍在四轮循环中）：

```
            map<I, int> sweep;
            for (int i : id)
            {
                auto it = sweep.lower_bound(-p[i].second);
                while (it != sweep.end())
                {
                    int j = it->second;
                    I dx = p[i].first - p[j].first;
                    I dy = p[i].second - p[j].second;
                    if (dx < dy) break;
                    e.emplace_back(dx + dy, i, j);
                    it = sweep.erase(it);
                }
                sweep[-p[i].second] = i;
            }
            for (auto &[x, y] : p)
                if (dir & 1) x = -x;
                else swap(x, y);
        }
        return e;
    }

    static Result solve(const vector<Point> &points)
    {
        auto e = candidates(points);
        int n = int(points.size());
        Result r;
        if (n < 2) return r;
        sort(e.begin(), e.end());
        dsu uf(n);
        for (auto [w, u, v] : e)
        {
            int a = uf.find(u), b = uf.find(v);
            if (a == b) continue;
            if (uf.size(a) < uf.size(b)) swap(a, b);
            uf.merge(a, b);
            r.weight += w;
            r.edges.emplace_back(u, v);
            if (r.edges.size() == points.size() - 1) break;
        }
        assert(r.edges.size() == points.size() - 1);
        return r;
    }
};
```

模板题使用：Library Checker Manhattan MST

题意：1≤N≤200000，N 个点的坐标 0..10^9，可重合且每个输入位置仍是不同顶点。完整图边权为曼哈顿距离；输出最小总权及 N−1 条 0-based 原编号边，任意最优树均可。结果 weight 为 128 位；本题粗上界 2e9 乘以 N−1，严格小于 4e14，所以此驱动才可转 long long 打印。N=1 输出 0、不输出边；库的空输入和有符号 64 位坐标另属 API 扩展。

所需模板：dsu（第 213 页，D.1.1 节）、ManhattanMST（第 67 页，4.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/manhattanmst

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<ManhattanMST::Point> p(n);
    for (auto &[x, y] : p) cin >> x >> y;
    auto r = ManhattanMST::solve(p);
    cout << (long long)r.weight << '\n';
    for (auto [u, v] : r.edges) cout << u << ' ' << v << '\n';
}
```

4.1.3 严格与非严格次小生成树、换边方案

问题描述：给定带权无向图，求最小生成树，以及次小生成树的权值：严格模式要求权值严格大于 MST，非严格模式只要求边集不同。同时给出由 MST 换一条边得到次小树的方案（换入边 `in`、换出边 `out`）。

核心思路：次小生成树总能由 MST 加入一条非树边 (u, v, w)、删去树上 u 到 v 路径上的一条边得到。先 Kruskal 求出 MST 并建树，倍增表 `mx[j][u]` 保存向上 2^j 条边中权值互不相同的最大与次大边编号（`merge`）。对每条非树、非自环边，查询路径上的这两条边：非严格时删最大边；严格时若最大边与 w 等权就改删次大边，不存在则跳过。在所有候选中取 MST $+ w - w_{\text{out}}$ 的最小值。

代码依赖：dsu（第 213 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：SecondMST(n)，n≥1，点号 0..n-1。add(u,v,w) 返回 0-based 边号；允许自环、重边、完整 signed64 位权。solve(strict=true) 返回 connected、128 位 MST 总权 weight、所选边号 tree，以及可选 128 位次小权 next 和换入/换出边号 in/out。strict=true 要求权值严格增大；false 要求边集与返回的 MST 不同，可同权。不连通返回默认空结果；无次小树时 next 为空且 in/out=-1。存在时将 tree 中的 out 删去并加入 in 即得方案。可加边后重算；每次 solve 重建。依赖 dsu，沿用路径减半及 y 根挂 x 根；不用按秩合并。时间 O((n+m)log(n+m))，空间 O(m+n log n)，DFS 递归深度 O(n)。merge/dfs/path 与工作数组为内部实现；原始 e 只读。

```
struct SecondMST
{
    using I = __int128_t;
    using Pair = array<int, 2>;

    struct Edge
    {
        int u, v;
        long long w;
    };

    struct Result
    {
        bool connected = false;
        I weight = 0;
        vector<int> tree;
        optional<I> next;
        int in = -1, out = -1;
    };

    int n, h;
    vector<Edge> e;
    vector<vector<pair<int, int>>> g;
    vector<int> dep;
    vector<vector<int>> up;
    vector<vector<Pair>> mx;

    SecondMST(int n) : n(n), h(bit_width((unsigned)n)) { assert(n > 0); }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        e.push_back({u, v, w});
        return (int)e.size() - 1;
    }
```

不同权摘要与递归预处理（接上页同一结构体）：

```
    // Keep edge IDs of the two largest DISTINCT weights; -1 means absent.
    Pair merge(Pair a, Pair b) const
    {
        for (int x : b)
        {
            if (x == -1) continue;
            if (a[0] == -1 || e[x].w > e[a[0]].w)
            {
                a[1] = a[0];
                a[0] = x;
            }
            else if (e[x].w != e[a[0]].w && (a[1] == -1 || e[x].w > e[a[1]].w))
                a[1] = x;
        }
        return a;
    }

    void dfs(int u, int p)
    {
        for (int j = 1; j < h; j++)
        {
            int v = up[j - 1][u];
            up[j][u] = up[j - 1][v];
            mx[j][u] = merge(mx[j - 1][u], mx[j - 1][v]);
        }
        for (auto [v, id] : g[u])
            if (v != p)
            {
                dep[v] = dep[u] + 1;
                up[0][v] = u;
                mx[0][v] = {id, -1};
                dfs(v, u);
            }
    }
```

路径查询（接上页同一结构体）：

```
72      Pair path(int u, int v) const
73      {
74          Pair ans{-1, -1};
75          if (dep[u] < dep[v]) swap(u, v);
76          for (int j = h - 1; j >= 0; j--)
77              if ((dep[u] - dep[v]) >> j & 1)
78              {
79                  ans = merge(ans, mx[j][u]);
80                  u = up[j][u];
81              }
82          if (u == v) return ans;
83          for (int j = h - 1; j >= 0; j--)
84              if (up[j][u] != up[j][v])
85              {
86                  ans = merge(ans, merge(mx[j][u], mx[j][v]));
87                  u = up[j][u];
88                  v = up[j][v];
89              }
90          return merge(ans, merge(mx[0][u], mx[0][v]));
91      }
```

Kruskal 与逐边换入（接上页同一结构体）：

```
    Result solve(bool strict = true)
    {
        vector<int> ids(e.size());
        iota(ids.begin(), ids.end(), 0);
        sort(ids.begin(),
             ids.end(),
             [&](int a, int b) { return tie(e[a].w, a) < tie(e[b].w, b); });
        dsu ds(n);
        vector<bool> used(e.size());
        g.assign(n, {});
        Result ans;
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (!ds.merge(u, v)) continue;
            used[id] = true;
            ans.weight += (I)w;
            ans.tree.push_back(id);
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        if ((int)ans.tree.size() != n - 1) return {};
        ans.connected = true;
        dep.assign(n, 0);
        up.assign(h, vector<int>(n));
        mx.assign(h, vector<Pair>(n, Pair{-1, -1}));
        dfs(0, -1);
        for (int id : ids)
        {
            auto [u, v, w] = e[id];
            if (used[id] || u == v) continue;
            auto p = path(u, v);
            int out = p[0];
            if (strict && e[out].w == w) out = p[1];
            if (out == -1) continue;
            I cost = ans.weight + (I)w - (I)e[out].w;
            if (!ans.next || cost < *ans.next)
            {
                ans.next = cost;
                ans.in = id;
                ans.out = out;
            }
        }
        return ans;
    }
};
```

4.1.4 Kruskal 最小生成树与生成森林

问题描述：给定无向带权图，求最小生成树；图不连通时求最小生成森林（每个连通块一棵最小生成树），给出总权、所选边号和连通块数。

核心思路：把所有边按权值升序稳定排序（同权按编号），依次考察：若两端点在并查集中不连通，就选这条边并合并，否则跳过（会成环）。每选一条边连通块数减一，因此最终 `cnt` 就是连通块数，为 1 表示整图连通、得到的是最小生成树。

代码依赖：dsu（第 213 页，D.1.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：Kruskal(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 加入无向边并返回 0-based 原边号，边权为完整 signed64，支持重边、自环和负权，边数 ≤INT_MAX。run() 每次从全部原边重建最小生成森林，返回连通块数；n>0 时返回 1 表示存在 MST，空图返回 0。weight 为 int128 总权，ids 为所选原边号，按 (权值, 原编号) 排序；e 保持插入顺序。加边后必须重新 run，旧结果不代表新图。支持重复运行和独立值复制。

依赖普通 dsu，调用方先把较大根放在前，保留 dsu 的 fa[y]=x 语义。时间 O(n+m log(m+1)+m α(n+1))，空间 O(n+m)；add 均摊 O(1)。总权绝对值 <2^94，不能无条件转 long long。自环不入树，断开图仍返回各块最小树；不使用 −1 总权判断失败，合法 MST 总权也可为 −1。成员只读，n 和 e 不直接修改。

```
struct Kruskal
{
    struct Edge
    {
        int u, v;
        long long w;
    };
    int n;
    vector<Edge> e;
    vector<int> ids;
    long long weight = 0;

    Kruskal(int n) : n(n) {}

    int add(int u, int v, long long w)
    {
        e.push_back({u, v, w});
        return int(e.size()) - 1;
    }

    // Minimum spanning forest on 0..n-1; ties by edge id. Returns the component count.
    int run()
    {
        vector<int> order(e.size());
        iota(order.begin(), order.end(), 0);
        stable_sort(order.begin(), order.end(), [&](int a, int b) { return e[a].w < e[b].w; });
        dsu d(n);
        ids.clear();
        weight = 0;
        int cnt = n;
        for (int id : order)
```

```
32              if (d.merge(e[id].u, e[id].v))
33              {
34                  ids.push_back(id);
35                  weight += e[id].w;
36                  cnt--;
37              }
38          return cnt;
39      }
40  };
```

模板题使用：Luogu P3366 / 最小生成树

题意：无向图求最小生成树总权，不连通输出 orz；n≤5000、m≤200000、边权 1..10000，允许自环重边。输入 1-based 点号减一，run()!=1 表示断连；本题总权可安全转 long long。

所需模板：dsu（第 213 页，D.1.1 节）、Kruskal（第 74 页，4.1.4 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3366

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    Kruskal g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    if (g.run() != 1)
    {
        cout << "orz\n";
    }
    else
    {
        cout << (long long)g.weight << '\n';
    }
}
```

4.1.5 稠密图 Prim 最小生成森林

问题描述：给定稠密无向带权图（邻接矩阵），求最小生成树或最小生成森林的总权，以及每个点在树中的父点。

核心思路：`d[v]` 是未选点 v 到已选点集的最轻边权，`pre[v]` 为该边另一端。共 n 步，每步线性扫描选出 `d` 最小的未选点 u：若 $d[u]=\infty$，说明剩余点与已选点集不连通，u 作为新树的根，连通块数加一；否则把 $d[u]$ 计入总权。然后用 u 的邻接矩阵行更新其余未选点的 `d` 和 `pre`。

接口契约：Prim(n)，0≤n≤INT_MAX，点号 0..n-1；add(u,v,w) 自动加无向边，允许完整 signed64 权、零权、负权、自环，重边只保留最小权。不保存原边编号；cost 为 int128 矩阵，inf=2^126 表示缺边，−1 和 LLONG_MAX 均为合法边权。

run() 每次重建最小生成森林，返回原图连通块数。n>0 时返回 1 表示 MST，空图返回 0。weight 为 int128 总权；pre[v] 为父点，根为 −1，非根选边权为 cost[pre[v]][v]。父点森林无环，支持断连和孤立点；同权任选，不承诺字典序。加边后须重新 run，支持重复运行与独立复制；成员只读。

构造和存储 $O(n^2)$，add 为 $O(1)$，run 为 $O(n^2)$，矩阵约 $16n^2$ 字节，按实际内存选择 n；稀疏图优先 Kruskal。每步选到已选点集的最轻边；所有剩余点均无跨边时开启新树，不把 inf 计入总权。只向未选点严格更新父点，负自环不入树。总权绝对值 <2^94，可为 −1 或超出 long long，不能用权值判断断连。

```
struct Prim
{
    static constexpr long long inf = LLONG_MAX / 4;
    int n;
    vector<vector<long long>> cost;
    vector<int> pre;
    long long weight = 0;

    Prim(int n) : n(n), cost(n, vector<long long>(n, inf)) {}

    void add(int u, int v, long long w) { cost[u][v] = cost[v][u] = min(cost[u][v], w); }

    // O(n^2) minimum spanning forest on 0..n-1; returns the component count.
    int run()
    {
        vector<long long> d(n, inf);
        vector<bool> vis(n);
        pre.assign(n, -1);
        weight = 0;
        int cnt = 0;
        for (int step = 0; step < n; step++)
        {
            int u = -1;
            for (int v = 0; v < n; v++)
                if (!vis[v] && (u == -1 || d[v] < d[u])) u = v;
            if (d[u] == inf)
                cnt++;
            else
                weight += d[u];
            vis[u] = true;
            for (int v = 0; v < n; v++)
                if (!vis[v] && cost[u][v] < d[v]) d[v] = cost[u][v], pre[v] = u;
        }
        return cnt;
    }
};
```

模板题使用：AOJ ALDS1_12_A / Minimum Spanning Tree

题意：输入 n 及 n×n 对称邻接矩阵，−1 表示缺边，其余为 0..2000；n≤100 且保证连通。只读取上三角的有效边，输出 MST 总权；本题可安全转 long long。注意输入 −1 缺边是题目约定，Prim.add 的 −1 是合法负权。

所需模板：Prim（第 77 页，4.1.5 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=ALDS1_12_A&lang=en

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    Prim g(n);
    for (int u = 0; u < n; u++)
        for (int v = 0; v < n; v++)
        {
            int w;
            cin >> w;
            if (u < v && w != -1) g.add(u, v, w);
        }
    g.run();
    cout << (long long)g.weight << '\n';
}
```

4.1.6 曼哈顿最小生成树：闭锥扫描与平局割证明

接口见第 4.1.2 节（第 67 页）。边权 $d(p,q)=|p_x-q_x|+|p_y-q_y|$。每点只保留四个向右闭锥中的最近邻，再对候选边 Kruskal；候选图含至少一棵 MST，但不含所有最优边，禁边等额外约束不能套用。

单方向锥 $q_x-p_x\ge q_y-p_y\ge 0$：按 $(x+y,\text{原编号})$ 排序，活动点以 $-y$ 为 map 键，首个满足条件者即最近邻，找到后删除。四轮坐标 $(x,y),(y,x),(-y,x),(x,-y)$。总时间 $O(n\log n)$，空间 $O(n)$。重复坐标是不同顶点，按编号生成零权链。

输入为 signed 64 位，先转 signed 128 位再做和差；单边最大 $2^{65}-2$。返回 0 基编号与 128 位总权；空集与单点返回 0 和空边集。

4.1.7 严格次小生成树与换边

`SecondMST` 先 Kruskal 求 MST，倍增维护树路径上两个不同的最大边权及边号。每条非树非自环边换入：非严格删路径最大边；严格且换入边等于最大值时删第二大不同权边，无则跳过。`tree` 删 `out`、加 `in` 得 `next`；无次小树时 `next` 为空，断图时 `connected=false`。总权用 int128，边号 -1 表示缺项。

4.2 最小树形图

4.2.1 有向最小树形图

问题描述：最小树形图：给定有向带权图和根 r，选出一些边，使从 r 出发能到达所有点且每个非根点恰有一条入边，求最小总权；有点不可达时无解。

核心思路：朱–刘算法。每轮为每个非根点选最小入边（忽略自环），先把这些边权全部计入答案；若某点没有入边则无解。沿 `pre` 从每个点往回走，若回到本轮已走过的点就找到一个环，给环上点分配同一编号 `id`；没有环时所选入边就是最优树形图。有环则把每个环缩成一个点，其余点各占一个新编号，每条边的权减去其原终点所选入边的权（选它进入环就要替换掉环上原来的那条入边），然后在缩小的图上重复。

思路中的情况	代码中的处理	行
每个非根点选最小入边	记录入边权 `in` 与起点 `pre`，自环不参与	21
计入答案或判无解	某非根点无入边则返回 `nullopt`，否则累加全部 `in`	29
沿入边回溯找环	用 `vis` 标记本次出发点，遇到已标记的点即在环上	36
无环	所选入边构成树形图，直接返回累计答案	48
缩环并修改边权	端点换成新编号，跨分量边减去原终点的 `in`	56

接口契约：solve(n,root,edges)，点号 0..n-1，n≥1、0≤root<n；返回以 root 为根向外可达的最小树形图费用，无解 nullopt。允许负权、重边和自环（自环不选），n=1 的费用为 0；费用 -1 与无解可区分。输入、缩环后边权及累计费用绝对值须小于 2^120，类型为 int128。每个非根点选最小入边；若成环则缩点，跨环入边减去原终点已选入边费用，以计算替换代价。O(VE)，edges 按值传入，原图保持不变。仅返回费用，不恢复边方案。双版本已做所有根的入边枚举与 800 点连续缩环核验，暂无专门 OJ AC。

```cpp
struct Arborescence
{
    using I = __int128_t;

    struct Edge
    {
        int u, v;
        I w;
    };

    // Directed minimum rooted OUT-arborescence cost, 0-based, or nullopt.
    static optional<I> solve(int n, int root, vector<Edge> e)
    {
        const I inf = I(1) << 120;
        I ans = 0;
        while (true)
        {
            vector<I> in(n, inf);
            vector<int> pre(n, -1), id(n, -1), vis(n, -1);
            for (auto a : e)
                if (a.u != a.v && a.w < in[a.v])
                {
                    in[a.v] = a.w;
                    pre[a.v] = a.u;
                }
            in[root] = 0;
            for (I x : in)
            {
                if (x == inf) return nullopt;
                ans += x;
            }
```

判环与缩点（接上页同一函数）：

```
            int cnt = 0;
            for (int i = 0; i < n; i++)
            {
                int u = i;
                while (vis[u] != i && id[u] == -1 && u != root)
                {
                    vis[u] = i;
                    u = pre[u];
                }
                if (u != root && id[u] == -1)
                {
                    id[u] = cnt;
                    for (int v = pre[u]; v != u; v = pre[v]) id[v] = cnt;
                    ++cnt;
                }
            }
            if (!cnt) return ans;
            for (int i = 0; i < n; i++)
                if (id[i] == -1) id[i] = cnt++;
            for (auto &a : e)
            {
                int v = a.v;
                a.u = id[a.u];
                a.v = id[a.v];
                if (a.u != a.v) a.w -= in[v];
            }
            root = id[root];
            n = cnt;
        }
    }
};
```

模板题使用：Luogu P4716

题意：给定根的有向最小树形图费用，无解输出-1。n 至 100、m 至 10^4、正权至 10^6；顶点和根从 1-based 转 0-based。optional 有值时才解引用，本题总费用可转 long long；核心只返回费用，不恢复选边。

所需模板：Arborescence（第 80 页，4.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P4716

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    cin >> n >> m >> r;
    vector<Arborescence::Edge> e;
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        e.push_back({u - 1, v - 1, w});
    }
    auto ans = Arborescence::solve(n, r - 1, e);
    // This problem's answer fits in long long.
    cout << (ans ? (long long)*ans : -1) << '\n';
}
```

第五章　斯坦纳树

5.1　最小斯坦纳树与原边方案

问题描述：给定非负权无向图和若干关键点（终端），求连通全部关键点的边权和最小的边集（最小斯坦纳树），并给出所选原边编号。

核心思路：状态 $dp[S][v]$ 为连通终端子集 S、且包含点 v（视为根）的树的最小代价，初值为单个终端在自身处代价 0。按 `mask` 从小到大处理：先在同一点 v 合并 S 的两个互补非空子集 T 与 $S \setminus T$（每对只枚举一次）；再以全部 $dp[S][\cdot]$ 为初值做多源 Dijkstra，让根沿边移动。转移来源分别记在 `split`（合并用的子集）与 `last`（最后走的边）。答案为 $\min_v dp[\text{全集}][v]$，从该根递归回溯标记用到的边，`seen` 防止重复展开同一状态。

思路中的情况	代码中的处理	行
子集合并	枚举真子集 `sub`，只处理 `sub` 不大于补集的一半，逐点相加更新并记 `split`	52
移动根	多源 Dijkstra 沿边松弛同一 `mask` 层，记下最后一条边 `last`	79
选最优根	全集层取最小值，等于 `inf` 说明终端不连通	85
回溯合并状态	拆成两个子集分别在同一点继续回溯	96
回溯移动状态	标记边 `id`，走到边的另一端继续回溯	102

接口契约：SteinerTree(n)，点号 0..n-1；add(u,v,w) 添加无向边并返回 0-based 原边编号，允许重边、自环、零权。n≥1，w≥0，所有边权总和严格小于 LLONG_MAX/4。solve(terminals) 自动去重，返回 {cost,edges}，edges 为不重复、升序原边编号；空终端返回 0 及空方案，无法连通返回-1 及空方案。输入图不修改，可重复求解；恢复最优连通子图，不额外保证零权环树化，递归回溯。

令 k 为去重终端数。dp[mask][v] 先枚举互补非空子集在 v 合并，再多源 Dijkstra 移动根。时间 O(3^k n+2^k(m+n)log n)，空间 O(2^k n+n+m)。位掩码约束 k≤20，不代表此规模必然能在时限内完成；P6192 为 n≤100、m≤500、k≤10。负权不适用。当前已通过独立边子集枚举和方案校验，P6192 已 100 分通过，排名未核验。

正确性要点　非负权最优连通子图可删环成树。以 v 为根，分叉处将关键点分成两个非空集合，在同根合并；单分支路径由最短路扩展得到。真子集先处理，逐 mask 闭包即可。回溯合并严格缩小 mask，同 mask 前驱遵循 Dijkstra 的确定次序；严格改善避免零权前驱环。原边去重后的连通方案费用不大于 dp，最优性又排除严格更小，故边权和恰等于 cost。

图存储与加边：

```
struct SteinerTree
{
    using ll = long long;
    static constexpr ll inf = LLONG_MAX / 4;
    struct Edge
    {
        int u, v;
        ll w;
    };
    struct Result
    {
        ll cost;
        vector<int> edges;
    };
    int n;
    ll total = 0;
    vector<Edge> edges;
    vector<vector<pair<int, int>>> g;

    explicit SteinerTree(int n) : n(n), g(n)
    {
        assert(n >= 1);
    }
    int add(int u, int v, ll w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        assert(0 <= w && w < inf - total);
        int id = edges.size();
        edges.push_back({u, v, w});
        g[u].push_back({v, id});
        g[v].push_back({u, id});
        total += w;
        return id;
    }
```

状态初始化与同根合并（接上页结构体）：

```
    Result solve(vector<int> terminals) const
    {
        sort(terminals.begin(), terminals.end());
        terminals.erase(unique(terminals.begin(), terminals.end()), terminals.end());
        for (int v : terminals) assert(0 <= v && v < n);
        int k = terminals.size();
        assert(k <= 20);
        if (!k) return {0, {}};
        int count = 1 << k;
        vector<vector<ll>> dp(count, vector<ll>(n, inf));
        vector<vector<int>> split(count, vector<int>(n));
        vector<vector<int>> last(count, vector<int>(n, -1));
        for (int i = 0; i < k; i++) dp[1 << i][terminals[i]] = 0;
        for (int mask = 1; mask < count; mask++)
        {
            for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask)
            {
                int other = mask ^ sub;
                if (sub > other) continue;
                for (int v = 0; v < n; v++)
                {
                    ll value = dp[sub][v] + dp[other][v];
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = sub;
                    }
                }
            }
```

多源最短路（接上页 mask 循环）：

```
            priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<pair<ll, int>>> pq;
            for (int v = 0; v < n; v++)
                if (dp[mask][v] < inf) pq.push({dp[mask][v], v});
            while (!pq.empty())
            {
                auto [distance, u] = pq.top();
                pq.pop();
                if (distance != dp[mask][u]) continue;
                for (auto [v, id] : g[u])
                {
                    ll value = distance + edges[id].w;
                    if (value < dp[mask][v])
                    {
                        dp[mask][v] = value;
                        split[mask][v] = 0;
                        last[mask][v] = id;
                        pq.push({value, v});
                    }
                }
            }
        }
```

最优根与递归方案恢复（接上页 solve 函数）：

```
        int root = min_element(dp.back().begin(), dp.back().end()) - dp.back().begin();
        if (dp.back()[root] == inf) return {-1, {}};
        vector<vector<char>> seen(count, vector<char>(n));
        vector<char> used(edges.size());
        auto trace = [&](auto &&self, int mask, int u) -> void
        {
            if (seen[mask][u]) return;
            seen[mask][u] = 1;
            int sub = split[mask][u];
            if (sub)
            {
                self(self, sub, u);
                self(self, mask ^ sub, u);
            }
            else if (last[mask][u] != -1)
            {
                int id = last[mask][u];
                used[id] = 1;
                self(self, mask, edges[id].u ^ edges[id].v ^ u);
            }
        };
        trace(trace, count - 1, root);
        Result result{dp.back()[root], {}};
        for (int id = 0; id < (int)edges.size(); id++)
            if (used[id]) result.edges.push_back(id);
        return result;
    }
};
```

模板题使用：P6192 最小斯坦纳树

题意：输入无向正权图及关键点，输出连接所有关键点的最小边权和。方案边编号可直接读取 solve 返回值的 edges。

所需模板：SteinerTree（第 84 页，5.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6192

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, k;
    cin >> n >> m >> k;
    SteinerTree tree(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        tree.add(u - 1, v - 1, w);
    }
    vector<int> terminals(k);
    for (int &v : terminals)
    {
        cin >> v;
        v--;
    }
    cout << tree.solve(terminals).cost << '\n';
}
```

第六章　连通性相关

6.1　强连通分量

相关应用：函数图包含尾链与环分解，导航无独立页；需正文定位，不应等同一般 SCC 算法。

6.1.1　函数图分解、跳转与到达

功能：给定每个点恰有一条出边的函数图（若干基环内向树），分解出所有环，支持查询从 u 出发走 k 步到达的点（k 可达 64 位），以及从 u 首次到达 v 的步数。

核心思路：先按入度剥叶得到 `order`，剩下入度非零的点都在环上，沿后继走一圈记下每个环 `cycles`、环编号 `component` 与环上位置 `position`。再逆序遍历 `order`，由后继推出树点的入环点 `entry` 与入环步数 `depth`。走 k 步时，若 k 不小于入环步数，先走到入环点，剩余步数对环长取模直接查环；否则 $k < n$，用倍增表 `up` 跳。`steps(u,v)`：v 不在环上时，检查 u 走深度差步是否恰好到 v；v 在环上时，答案为入环步数加上环上从 `entry[u]` 到 v 的距离。

接口契约：FunctionalGraph(to) 或 build(to) 重建，每个点恰有一条出边，to[u] 为后继，点号 0..n-1；空图允许，但不能查询。advance(u,k) 返回第 k 步到达的点，k 为完整 unsigned long long；steps(u,v) 返回首次到达 v 的最少步数，无法到达返回 -1，自身为 0。component[u] 为所在环的编号；cycles[id] 按后继方向保存环；entry[u] 为首次入环点；depth[u] 为入环步数；position 仅在环点上有效，非环点为 -1。order 为入度剥离顺序，逆序可用于从后继向入环树传播状态；up 是倍增表。所有字段只读。O(n log n) 预处理及空间，入环前查询 O(log n)，入环后取模 O(1)，不为 64 位步数分配 64 层表。默认构造为空图，支持自环、多个分量及重建。n≤INT_MAX，实际规模须满足内存。

```
struct FunctionalGraph
{
    using U = unsigned long long;
    int n;
    vector<int> component, position, entry, depth, order;
    vector<vector<int>> cycles, up;

    explicit FunctionalGraph(vector<int> to = {}) { build(move(to)); }

    void build(vector<int> to)
    {
        assert(to.size() <= INT_MAX);
        n = to.size();
        vector<int> degree(n);
        for (int v : to)
        {
            assert(0 <= v && v < n);
            degree[v]++;
        }
        order.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] == 0) order.push_back(u);
        for (int i = 0; i < int(order.size()); i++)
        {
            int v = to[order[i]];
            if (--degree[v] == 0) order.push_back(v);
        }
```

环、入环树与倍增表（接上页同一结构体与函数）：

```
        component.assign(n, -1);
        position.assign(n, -1);
        entry.resize(n);
        depth.assign(n, 0);
        cycles.clear();
        for (int u = 0; u < n; u++)
            if (degree[u] && component[u] == -1)
            {
                vector<int> cycle;
                int v = u, id = cycles.size();
                do
                {
                    component[v] = id;
                    position[v] = cycle.size();
                    entry[v] = v;
                    cycle.push_back(v);
                    v = to[v];
                } while (v != u);
                cycles.push_back(move(cycle));
            }
        for (int i = int(order.size()) - 1; i >= 0; i--)
        {
            int u = order[i], v = to[u];
            component[u] = component[v];
            entry[u] = entry[v];
            depth[u] = depth[v] + 1;
        }
        up.assign(1, to);
        for (int k = 1; (1ULL << k) <= U(n); k++)
        {
            vector<int> row(n);
            for (int u = 0; u < n; u++) row[u] = up[k - 1][up[k - 1][u]];
            up.push_back(move(row));
        }
    }
```

跳转与最少步数接口（接上页同一结构体）：

```
    int advance(int u, U k) const
    {
        assert(0 <= u && u < n);
        if (k >= U(depth[u]))
        {
            k -= depth[u];
            u = entry[u];
            const auto &cycle = cycles[component[u]];
            return cycle[(U(position[u]) + k % cycle.size()) % cycle.size()];
        }
        for (int bit = 0; k; bit++, k >>= 1)
            if (k & 1) u = up[bit][u];
        return u;
    }

    int steps(int u, int v) const
    {
        assert(0 <= u && u < n && 0 <= v && v < n);
        if (component[u] != component[v] || depth[u] < depth[v]) return -1;
        if (depth[v] > 0)
        {
            int d = depth[u] - depth[v];
            return advance(u, d) == v ? d : -1;
        }
        int offset = position[v] - position[entry[u]];
        if (offset < 0) offset += int(cycles[component[u]].size());
        return depth[u] + offset;
    }
};
```

模板题使用：CSES 1750 Planets Queries I

题意：给每点唯一后继，查询从 x 走 k 步后的点，n,q≤200000，0≤k≤10^9。题目 1-based 点号读入减 1、输出加 1；k 用 unsigned long long，模板也支持完整 64 位步数。advance 先处理入环链再对环长取模，不能只建 30 层就宣称支持任意 64 位。预处理 O(n log n)，查询 O(log n)，入环后 O(1)。

所需模板：FunctionalGraph（第 90 页，6.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://cses.fi/problemset/task/1750

```
int main()
{
    int n, q;
    scanf("%d%d", &n, &q);
    vector<int> to(n);
    for (auto &v : to)
    {
        scanf("%d", &v);
        v--;
    }
    FunctionalGraph graph(to);
    while (q--)
    {
        int u;
        unsigned long long k;
        scanf("%d%llu", &u, &k);
        printf("%d\n", graph.advance(u - 1, k) + 1);
    }
}
```

6.1.2 Tarjan 强连通分量与缩点

问题描述：给定有向图，求每个点所属的强连通分量编号，并构造缩点后的 DAG（邻接表）。

核心思路：DFS 时记录时间戳 `dfn` 与 `low`（能回溯到的仍在栈中的最小时间戳），访问过的点压入栈 `st`。树边用子节点的 `low` 更新；指向已访问且尚未分配分量的点的边，用其 `dfn` 更新。回溯时若 $\mathrm{low}[u]=\mathrm{dfn}[u]$，$u$ 是分量的根，弹栈到 u 为止得到一个分量。分量按完成先后编号，先完成者无出边指向后完成者，因此跨分量边总从大编号指向小编号，编号即逆拓扑序。

接口契约：TarjanSCC(n)，点号 1..n；add(u,v) 加有向边，run() 从头计算。bel[u] 为 1..cnt 的强连通分量编号，跨分量边总从大编号指向小编号，可按 cnt..1 做拓扑 DP。dag() 返回缩点邻接表，保留重边；若 DP 需要对不同邻点去重，调用者应排序去重。支持自环、重边、不连通图和重复 run。时间及图存储 O(n+m)，递归 DFS 另需 O(n) 调用栈；本地验证应配置足够栈内存。注意已有 SCC 模板的编号方向与本模板相反。

```
struct TarjanSCC
{
    int n, timer = 0, cnt = 0;
    vector<vector<int>> g;
    vector<int> dfn, low, bel, st;

    TarjanSCC(int n) : n(n), g(n + 1) {}

    void add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        g[u].push_back(v);
    }
```

```
14
15      void dfs(int u)
16      {
17          dfn[u] = low[u] = ++timer;
18          st.push_back(u);
19          for (int v : g[u])
20          {
21              if (!dfn[v])
22              {
23                  dfs(v);
24                  low[u] = min(low[u], low[v]);
25              }
26              else if (!bel[v])
27                  low[u] = min(low[u], dfn[v]);
28          }
29          if (low[u] == dfn[u])
30          {
31              ++cnt;
32              while (true)
33              {
34                  int v = st.back();
35                  st.pop_back();
36                  bel[v] = cnt;
37                  if (v == u) break;
38              }
39          }
40      }
41
42      // IDs 1..cnt are in reverse topological order. Recursive DFS needs O(n) stack.
43      void run()
44      {
45          timer = cnt = 0;
46          dfn.assign(n + 1, 0);
47          low = dfn;
48          bel = dfn;
49          st.clear();
50          for (int u = 1; u <= n; u++)
51          {
52              if (!dfn[u]) dfs(u);
53          }
54      }
55
56      // Retains parallel edges. Use after run().
57      vector<vector<int>> dag() const
58      {
59          vector<vector<int>> result(cnt + 1);
60          for (int u = 1; u <= n; u++)
```

```
        {
            for (int v : g[u])
            {
                if (bel[u] != bel[v]) result[bel[u]].push_back(bel[v]);
            }
        }
        return result;
    }
};
```

模板题使用：Library Checker scc

题意：把有向图分成强连通分量，并按缩点图的拓扑顺序输出各分量。题目点号为 0..n-1，调用时加 1，输出时减 1。

所需模板：TarjanSCC（第 94 页，6.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/scc

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    TarjanSCC g(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        cin >> u >> v;
        g.add(u + 1, v + 1);
    }
    g.run();
    vector<vector<int>> c(g.cnt + 1);
    for (int u = 1; u <= n; u++) c[g.bel[u]].push_back(u - 1);
    cout << g.cnt << '\n';
    for (int i = g.cnt; i >= 1; i--)
    {
        cout << c[i].size();
        for (int u : c[i]) cout << ' ' << u;
        cout << '\n';
    }
}
```

6.1.3 SCC 模板题与速度榜口径

QOJ906 要求按拓扑序输出强连通分量。第 94 页 `TarjanSCC` 的分量编号为逆拓扑序，按 cnt 到 1 输出；`bel[1..n]` 为 1-based，0-based 输入点号先加一、输出时减一。

6.2 双连通分量

6.2.1 点双与边双：共享 lowlink 核心

问题描述：对无向图（可不连通，可有重边、自环）一次求出点双连通分量（各块的顶点集合与割点）和边双连通分量（桥以及每个点所属的边双编号）。

核心思路：一次 Tarjan DFS 维护 `dfn` 与 `low`，按边编号而不是父点跳过父边，所以重边能正确地阻止成桥。树边 $u \to v$ 回溯后：若 $low_v > dfn_u$，该边是桥；若 $low_v \geq dfn_u$，u 是一个点双的最高点，从栈 `st` 弹出直到 v，连同 u 组成一个块，并且非根点、或有两个以上 DFS 儿子的根是割点。孤立点单独成块。之后用不经过桥的 `paint` 做连通染色，每个连通部分就是一个边双，编号存入 `bel`。

接口契约：BiconnectedCore(n)，点号 1..n；add(u,v) 返回原边的 0-based 编号，run() 从头计算，允许重复运行。

点双结果：blocks 为点双的顶点列表，桥两端成块，孤立点单独成块，割点可出现在多个块；cut[u] 标记割点。

边双结果：bridge[id] 标记桥，bel[u] 是 1..cnt 的边双编号；cnt 是全图边双个数。重边按边号处理；自环保留边号但不进 DFS，不是桥，不额外产生点双。一次共享 DFS 后递归染色，O(n+m) 时间和空间，DFS 保持递归。n=0 合法。run 后才可读取结果或构造森林；g、edges 为原图，不在 DFS 期间修改。

```
struct BiconnectedCore
{
    int n, timer = 0, cnt = 0;
    vector<vector<pair<int, int>>> g;
    vector<pair<int, int>> edges;
    vector<vector<int>> blocks;
    vector<int> dfn, low, cut, bridge, bel, st;

    BiconnectedCore(int n) : n(n), g(n + 1) {}

    int add(int u, int v)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        int id = edges.size();
        edges.push_back({u, v});
        if (u != v)
        {
            g[u].push_back({v, id});
            g[v].push_back({u, id});
        }
        return id;
    }
```

```
    void dfs(int u, int parent_edge)
    {
        dfn[u] = low[u] = ++timer;
        st.push_back(u);
        int children = 0;
        for (auto [v, id] : g[u])
        {
            if (id == parent_edge) continue;
            if (!dfn[v])
            {
                children++;
                dfs(v, id);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) bridge[id] = 1;
                if (low[v] >= dfn[u])
                {
                    if (parent_edge != -1 || children > 1) cut[u] = 1;
                    blocks.push_back({u});
                    while (true)
                    {
                        int x = st.back();
                        st.pop_back();
                        blocks.back().push_back(x);
                        if (x == v) break;
                    }
                }
            }
            else
                low[u] = min(low[u], dfn[v]);
        }
        if (parent_edge == -1 && children == 0) blocks.push_back({u});
    }
```

```
    void paint(int u)
    {
        bel[u] = cnt;
        for (auto [v, id] : g[u])
        {
            if (!bridge[id] && !bel[v]) paint(v);
        }
    }
```

```
    void run()
    {
        timer = cnt = 0;
        dfn.assign(n + 1, 0);
        low = dfn;
        cut = dfn;
        bel = dfn;
        bridge.assign(edges.size(), 0);
        blocks.clear();
        st.clear();
        for (int u = 1; u <= n; u++)
        {
            if (!dfn[u])
            {
                dfs(u, -1);
                st.pop_back();
            }
        }
        for (int u = 1; u <= n; u++)
        {
            if (bel[u]) continue;
            ++cnt;
            paint(u);
        }
    }
};
```

模板题使用：Luogu P8435

题意：给定无向图，输出所有点双连通分量的顶点集合；割点可出现在多个集合中，孤立点单独成块。点号 1..n。

所需模板：BiconnectedCore（第 97 页，6.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P8435

```
int main()
{
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int n, m;
    std::cin >> n >> m;
    BiconnectedCore graph(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        std::cin >> u >> v;
        graph.add(u, v);
    }
    graph.run();
    const auto &blocks = graph.blocks;
    std::cout << blocks.size() << '\n';
    for (const auto &block : blocks)
    {
        std::cout << block.size();
        for (int u : block) std::cout << ' ' << u;
        std::cout << '\n';
    }
}
```

6.2.2 边双缩点森林（保留桥编号）

功能：由已运行的 `BiconnectedCore` 构造边双缩点森林：每个边双连通分量缩成一个点，桥成为森林的边，邻接项附带原桥编号。

核心思路：遍历全部原边，只保留桥 `bridge[id]`，把两个端点换成所属边双编号 `bel`，在两个分量之间加一条双向边。删去桥后各边双互不相连，所以缩点后每个连通分量是一棵树。

代码依赖：BiconnectedCore（第 97 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：bridge_component_forest(graph) 接收已 run 的 BiconnectedCore，返回按 1..cnt 编号的边双森林，邻接项为 (相邻分量, 原桥编号)。每条桥变成一条森林边，重边和自环不会误作桥。0 号空置，不连通输入可产生多棵树，孤立边双保留空邻接表。O(n+m) 时间、O(n) 输出空间，不修改分解结果。

```
vector<vector<pair<int, int>>> bridge_component_forest(const BiconnectedCore &graph)
{
    int cnt = graph.cnt;
    const auto &edges = graph.edges;
    const auto &bridge = graph.bridge;
    const auto &bel = graph.bel;
    vector<vector<pair<int, int>>> tree(cnt + 1);
    for (int id = 0; id < (int)edges.size(); id++)
    {
        if (!bridge[id]) continue;
        auto [u, v] = edges[id];
        u = bel[u];
        v = bel[v];
        tree[u].push_back({v, id});
        tree[v].push_back({u, id});
    }
    return tree;
}
```

6.3 圆方树

6.3.1 静态仙人掌森林最短路

问题描述：给定边权非负的仙人掌森林（每条边至多属于一个简单环），多次询问两点间最短路的长度，不连通时返回 -1。

核心思路：建圆方树：DFS 中遇到返祖边 $u \to v$，就为这个环新建虚点 b，环长记为 `len[b]`；环的最高点 v 以权 0 连向 b，环上其余点 x 以到 v 的较短弧长连向 b，`pos[x]` 记它沿 DFS 树到 v 的距离。不在环上的树边原样保留。这样树上根距离 `dis` 就等于原图中到根的最短距离。询问用倍增求 LCA p：p 是原点时答案为 $dis_u + dis_v - 2dis_p$；p 是环虚点时，u, v 跳到的两个儿子 u', v' 在同一环上，答案为 $dis_u - dis_{u'} + dis_v - dis_{v'}$ 加上 u', v' 之间的较短弧。

思路中的情况	代码中的处理	行
普通树边	儿子 v 不属于以上方为顶的环时，直接作为带权树边	36
发现环	返祖边新建环虚点，环长为两端前缀距离差加该边权	40
环上点挂到虚点	沿 `par` 从 u 回走到 v，权取两条弧中较短者	47
LCA 是原点	根距离之和减去两倍 LCA 根距离	111
LCA 是环虚点	用两个儿子在环上的位置差取较短弧，替换经过虚点的部分	112

接口契约：Cactus(n)，0≤n<INT_MAX/2，原点 1..n。add(u,v,w) 加入无向边；输入须为无自环无重边的仙人掌森林（每条边最多属于一个简单环），不检查此结构前提。权非负，所有边权之和 ≤LLONG_MAX/4。build() 递归构造桥/环虚点森林并倍增；distance(u,v) 仅接受原点，断连返回-1，同点返回 0。查询前必须 build；加边后必

须重建，支持重复 build 和复制，无动态修改或路径方案接口。编号 n+1..cnt 为环虚点；t 是从根向下的带权邻接表，成员只读。环的顶点到最上方割点距离取较短弧，虚点到该割点权 0；LCA 是虚点时，须用两个儿子在环上的较短弧替换根距离贡献。dfs 与 prepare 均递归，环上沿 par 回走仅枚举边，不模拟 DFS 栈。原图 DFS 及环枚举 O(n+m)，总预处理/空间 O((n+m)log(n+1))，查询 O(log(n+1))，递归深度 O(n)。n=0 允许 build 但无合法查询；不支持负权、非法仙人掌或修改内部数组。

```
struct Cactus
{
    using ll = long long;
    int n, m = 0, cnt, timer = 0, lg;
    vector<vector<tuple<int, int, ll>>> g;
    vector<vector<pair<int, ll>>> t;
    vector<vector<int>> up;
    vector<int> dfn, par, bel, dep, root;
    vector<ll> pref, pos, len, dis;

    Cactus(int n) : n(n), g(n + 1)
    {
        assert(0 <= n && n < INT_MAX / 2);
    }

    void add(int u, int v, ll w)
    {
        assert(1 <= u && u <= n && 1 <= v && v <= n);
        assert(u != v && w >= 0);
        g[u].push_back({v, m, w});
        g[v].push_back({u, m, w});
        m++;
    }
```

递归构造环虚点与根距离（接上页同一结构体）：

```
    void dfs(int u, int pe)
    {
        dfn[u] = ++timer;
        for (auto [v, id, w] : g[u])
        {
            if (id == pe) continue;
            if (!dfn[v])
            {
                par[v] = u;
                pref[v] = pref[u] + w;
                dfs(v, id);
                if (!bel[v]) t[u].push_back({v, w});
            }
            else if (dfn[v] < dfn[u])
            {
                int b = ++cnt;
                len[b] = pref[u] - pref[v] + w;
                t[v].push_back({b, 0});
                for (int x = u; x != v; x = par[x])
                {
                    bel[x] = b;
                    pos[x] = pref[x] - pref[v];
                    t[b].push_back({x, min(pos[x], len[b] - pos[x])});
                }
            }
        }
    }

    void prepare(int u)
    {
        for (int k = 1; k < lg; k++)
            up[k][u] = up[k - 1][up[k - 1][u]];
        for (auto [v, w] : t[u])
        {
            up[0][v] = u;
            dep[v] = dep[u] + 1;
            root[v] = root[u];
            dis[v] = dis[u] + w;
            prepare(v);
        }
    }
```

从零重建（接上页同一结构体）：

```
    void build()
    {
        cnt = n;
        timer = 0;
        int size = 2 * n + 1;
        t.assign(size, {});
        dfn.assign(size, 0);
        par = bel = dep = root = dfn;
        pref.assign(size, 0);
        pos = len = dis = pref;
        for (int u = 1; u <= n; u++)
            if (!dfn[u]) dfs(u, -1);
        lg = 1;
        while ((1LL << lg) <= cnt) lg++;
        up.assign(lg, vector<int>(size));
        for (int u = 1; u <= n; u++)
        {
            if (dfn[u] && !par[u])
            {
                root[u] = u;
                prepare(u);
            }
        }
    }
```

查询与环上距离修正（接上页同一结构体）：

```
    // Original vertices only; disconnected pairs return -1.
    ll distance(int u, int v) const
    {
        if (root[u] != root[v]) return -1;
        ll ans = dis[u] + dis[v];
        if (dep[u] < dep[v]) swap(u, v);
        int delta = dep[u] - dep[v];
        for (int k = 0; k < lg; k++)
            if (delta >> k & 1) u = up[k][u];
        if (u == v) return ans - 2 * dis[u];
        for (int k = lg - 1; k >= 0; k--)
        {
            if (up[k][u] != up[k][v])
            {
                u = up[k][u];
                v = up[k][v];
            }
        }
        int p = up[0][u];
        if (p <= n) return ans - 2 * dis[p];
        ll arc = abs(pos[u] - pos[v]);
        return ans - dis[u] - dis[v] + min(arc, len[p] - arc);
    }
};
```

为什么需要环上修正 桥和环块构成森林，块间路径唯一。若最近公共祖先是原点，两条根距离减去公共前缀即可；若是环虚点，u、v 现为该虚点的两个原点儿子，先扣掉这两点到根的距离，再加环上两段弧长的较小值。不能直接把虚点当普通 LCA，否则会强迫路径绕经环顶割点。

复杂度与前提 每条边只属于至多一个环，因此沿 par 回走的总次数为 O(m)，不是每个返祖边各走 O(n)。两次 DFS 均为递归。输入须为非负权简单仙人掌森林，全部边权和不超过 LLONG_MAX/4；不自动校验结构，非法图可能越界。加边后必须重建。

模板题使用：P5236 【模板】静态仙人掌

题意：正权无重边仙人掌，查询两点最短距离；n,q≤10000、m≤20000、w≤100000。

所需模板：Cactus（第 101 页，6.3.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P5236

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    cin >> n >> m >> q;
    Cactus solver(n);
    for (int i = 0; i < m; i++)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        solver.add(u, v, w);
    }
    solver.build();
    while (q--)
    {
        int u, v;
        cin >> u >> v;
        cout << solver.distance(u, v) << '\n';
    }
}
```

6.3.2 由点双构造圆方森林

功能：由已运行的 BiconnectedCore 的点双分解构造圆方树（森林）：原点为圆点，每个点双对应一个方点，返回邻接表。

核心思路：第 i 个点双的方点编号为 $n+i+1$，块内每个原点都与该方点连边。割点属于多个块，因此连向多个方点；原图两点间每条路径都必经的点，恰是它们在圆方树路径上的圆点。

代码依赖：BiconnectedCore（第 97 页，6.2.1 节）。抄写时一并准备；标准库类型不列入算法依赖。

接口契约：block_cut_forest(graph) 接收已经 run 的 BiconnectedCore，返回 1-based 邻接表。原点为 1..n，blocks[i] 的方点为 n+i+1；每个点双内的所有原点连向该方点。孤立点也连向自己的单点方块。不连通原图得到森林；0 号空置。O(n+m) 时间与输出空间，不修改分解结果。

```
vector<vector<int>> block_cut_forest(const BiconnectedCore &graph)
{
    int n = graph.n;
    const auto &blocks = graph.blocks;
    vector<vector<int>> tree(n + blocks.size() + 1);
    for (int i = 0; i < (int)blocks.size(); i++)
    {
```

```
        int b = n + i + 1;
        for (int u : blocks[i])
        {
            tree[u].push_back(b);
            tree[b].push_back(u);
        }
    }
    return tree;
}
```

第七章　环计数问题

7.1　简单无向图四元环计数

问题描述：给定简单无向图，统计四元环个数：四个不同点 a,b,c,d 满足 ab,bc,cd,da 都是边（不要求无弦）。例如 K_4 有 3 个四元环。

核心思路：按 (度数, 编号) 给所有点定一个全序，每个四元环只在其中最大的点 u 处计数。对每个 u，枚举二步路径 $u \to v \to w$，要求 v,w 都排在 u 之前；`cnt[w]` 记录已出现的从 u 到 w 的此类路径数，新路径与之前每条配对构成一个以 u,w 为对角的四元环，所以答案加 `cnt[w]`。处理完 u 后只清空 `touched` 中被访问过的 `cnt`。按度数定序保证总枚举量为 $O(m\sqrt{m})$。

接口契约：count_four_cycles(n,edges) 返回 long long 计数，n≥0、点 0..n-1、m=edges.size()≤INT_MAX。输入必须是简单无向图，每条边恰好给一次，不允许重边和自环；不自动去重。允许空图、孤立点、断连，输入不变，可重复调用。计四个不同点组成的四边环，不要求诱导子图无弦；K4 有 3 个四元环，旋转和反向不重复计算。按 (度数, 编号) 严格定序，以唯一最大点 u 为起点，统计 u-v-w 中 v,w 均小于 u 的二步路径；第二条路径与前面路径配对。每轮清空实际访问的 cnt。时间 O(n+m$\sqrt{\text{m}}$)，空间 O(n+m)，无需排序或递归；不可仅按编号定序后仍声称此复杂度。cnt 只需 int，答案必须 long long；在 m≤INT_MAX 约定下，环数 ≤m(m-1)/2<2^61，不溢出。

```
long long count_four_cycles(int n, const vector<pair<int, int>> &edges)
{
    assert(n >= 0 && edges.size() <= INT_MAX);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        g[u].push_back(v);
        g[v].push_back(u);
    }
    auto less = [&](int u, int v)
    {
        return pair{g[u].size(), u} < pair{g[v].size(), v};
    };
    vector<int> cnt(n), touched;
    long long ans = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            if (!less(v, u)) continue;
            for (int w : g[v])
            {
                if (!less(w, u)) continue;
```

```
                if (cnt[w] == 0) touched.push_back(w);
                ans += cnt[w];
                cnt[w]++;
            }
        }
        for (int w : touched) cnt[w] = 0;
        touched.clear();
    }
    return ans;
}
```

模板题使用：U367189 无向图四元环计数

题意：简单无向图，n≤100000，m≤200000；允许断连，不含重边和自环。输入 1-based 端点，输出非诱导四元环数。

所需模板：count_four_cycles（第 108 页，7.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/U367189

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges)
    {
        cin >> u >> v;
        u--;
        v--;
    }
    cout << count_four_cycles(n, edges) << '\n';
}
```

7.2 无向简单图三元环枚举

问题描述：给定简单无向图，对每个三元环（三个两两相邻的点）恰好调用一次回调 `visit(u,v,w)`，可用于计数、求权值和或收集方案。

核心思路：把每条边从（度数，编号）较小的端点指向较大的端点，得到无环有向图，每个三角形恰有一种 $u \to v \to w$ 且 $u \to w$ 的形态。枚举 u 时先把 u 的所有出邻点的 `mark` 设为 u，再枚举 $u \to v \to w$，若 w 带有标记就找到一个三角形。按度数定向保证每个点出度为 $O(\sqrt{m})$。

接口契约：enumerate_triangles(n,edges,visit) 使用 0..n-1 点号；edges 须为无向简单图，无自环、无重边，每条边只给一次，端点顺序任意。每个三角形恰调用一次 visit(u,v,w)，参数按度数和编号组成的次序排列，不保证编号递增。可在回调中计数、累计权值或收集方案；计数用 long long。按 (度数, 编号) 从小到大定向，标记 u 的出邻点后枚举 u 到 v 到 w 的有向路径。每个出度为 O($\sqrt{\text{m}}$)，总时间 O(n+m$\sqrt{\text{m}}$)，含常数时间回调，空间 O(n+m)；空图合法。调用期间不修改输入边集。

```cpp
template <class F>
void enumerate_triangles(int n, const vector<pair<int, int>> &edges, F visit)
{
    assert(n >= 0);
    vector<int> deg(n), mark(n, -1);
    vector<vector<int>> g(n);
    for (auto [u, v] : edges)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        deg[u]++, deg[v]++;
    }
    for (auto [u, v] : edges)
    {
        if (pair{deg[u], u} > pair{deg[v], v}) swap(u, v);
        g[u].push_back(v);
    }
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u]) mark[v] = u;
        for (int v : g[u])
            for (int w : g[v])
                if (mark[w] == u) visit(u, v, w);
    }
}
```

模板题使用：Library Checker enumerate_triangles

题意：给简单无向图的每个点一个权值，求所有三角形三个点权乘积之和，模 998244353。点号 0..n-1。

所需模板：enumerate_triangles（第 110 页，7.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://judge.yosupo.jp/problem/enumerate_triangles

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> x(n);
    for (auto &v : x) cin >> v;
    vector<pair<int, int>> edges(m);
    for (auto &[u, v] : edges) cin >> u >> v;
    const long long mod = 998244353;
    long long ans = 0;
    enumerate_triangles(n,
                        edges,
                        [&](int u, int v, int w)
                        { ans = (ans + x[u] * x[v] % mod * x[w]) % mod; });
    cout << ans << '\n';
}
```

第八章　最小环

相关应用：Karp 最小平均权有向环，目标不同于最小总权环；两遍 DP 线性空间。

8.1　有向图最小平均权环：线性空间 Karp

问题描述：给定带权有向图（权可负），求所有有向环中平均边权（环总权除以边数）的最小值，以未约分的分数形式返回；图中无环时返回空。

核心思路：Karp 定理：令 $F_k(v)$ 为恰好走 k 条边到达 v 的最短行走长度，起点任意（$F_0 \equiv 0$，相当于超级源），则最小平均权为 $\min_v \max_{0\le k<n} \frac{F_n(v)-F_k(v)}{n-k}$。为只用 $O(n)$ 空间，先用 `step` 滚动 n 轮求出 F_n 存入 `last`，再把 `d` 清零重新逐轮计算 $F_0,\dots,F_{n-1}$，同时对每个 v 维护分数最大值 `best[v]`，最后取各点中的最小者。分数比较用交叉相乘，整数权用 128 位保存。

接口契约：minimum_mean_cycle(n,edges)，0≤n<INT_MAX，点 0..n-1，edges 为 tuple<int,int,T> 有向边；支持断连、负权、自环和重边，输入不变。返回 optional<pair<W,int»，无环 nullopt；有环返回分子/正分母，未约分，不返回环方案。T 为有符号至多 64 位整数时 W=int128，全部比较精确，支持 LLONG_MIN/MAX；n<2^31 使交叉乘法绝对值 <2^126。实数 T 须支持值语义，W=T；禁止 NaN/Inf，要求 $2n^2\max|w|<2^{118}$ 避免溢出，仍有舍入误差，不能泛称精确。P3199 驱动用 GCC ___float128 进行 113 位核心运算，long double 仅用于输入输出；需 GCC 支持此扩展。F0 全部为 0，Fk 为恰好 k 条边的最短行走；先滚动到 Fn，再重算 F0..Fn-1，取 min_v max_k (Fn(v)-Fk(v))/(n-k)。时间 O(n(n+m))，额外 O(n)，输入 O(m)；非强连通也成立，无需 DFS、二分或负环启发式。k 为 0..n-1，不能只比较最后一层或允许至多 k 条边。

```
template <class T>
auto minimum_mean_cycle(int n, const vector<tuple<int, int, T>> &edges)
{
    static_assert(!is_integral_v<T> || (is_signed_v<T> && sizeof(T) <= 8));
    assert(0 <= n && n < INT_MAX);
    using W = conditional_t<is_integral_v<T>, __int128_t, T>;
    using Ratio = pair<W, int>;
    const W inf = W(1LL << 60) * W(1LL << 60);
    vector<W> d(n), next(n);
    auto step = [&]()
    {
        fill(next.begin(), next.end(), inf);
        for (auto [u, v, w] : edges)
        {
            if (d[u] == inf) continue;
            W value = d[u] + w;
            if (value < next[v]) next[v] = value;
        }
        d.swap(next);
    };
    for (int k = 0; k < n; k++) step();
    auto last = d;
    fill(d.begin(), d.end(), 0);
    auto less = [&](const Ratio &a, const Ratio &b)
    {
        return a.first * b.second < b.first * a.second;
    };
```

第二遍重算并取 min-max（接上页同一函数）：

```
    vector<Ratio> best(n, {0, 0});
    for (int k = 0; k < n; k++)
    {
        for (int v = 0; v < n; v++)
        {
            if (last[v] == inf || d[v] == inf) continue;
            Ratio now{last[v] - d[v], n - k};
            if (best[v].second == 0 || less(best[v], now))
                best[v] = now;
        }
        if (k + 1 < n) step();
    }
    optional<Ratio> ans;
    for (auto r : best)
    {
        if (r.second && (!ans || less(r, *ans))) ans = r;
    }
    return ans;
}
```

为什么全点初始化成立 把各边减去最小平均值后，所有环非负且存在零环。任意起点最短行走都能删环变成少于 n 条边的路径，所以每点的 max 比值非负。到零环的最短行走接无限重复零环，取长度 n 的最短前缀，其终点使 max 为 0；因此全局 min 为 0。恢复原权后所有比值增加同一常数，公式仍成立。无环图没有长度 n 行走，返回 nullopt。

返回分数与数值类型 分母为 1..n；不保证约分，也不保证分子本身就是某个环的权和。整数类型提升到 int128 进行运算和交叉比较；小数用调用者选择的 T，普通浮点不是精确计算。P3199 采用 GCC 的 113 位浮点进行核心运算，long double 仅用于输入输出。无需保存全部 DP 层，但第二遍必须重新从全 0 开始。

模板题使用：P3199 [HNOI2009] 最小圈

题意：实数有向边权，n≤3000、m≤10000、|w|≤10^7，保证有圈；GCC ___float128 核心运算，long double 读入/输出 8 位小数。

所需模板：minimum_mean_cycle（第 112 页，8.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P3199

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    using Real = __float128;
    int n, m;
    cin >> n >> m;
    vector<tuple<int, int, Real>> edges;
    while (m--)
    {
        int u, v;
        long double w;
        cin >> u >> v >> w;
        edges.emplace_back(u - 1, v - 1, w);
    }
    auto ans = minimum_mean_cycle(n, edges);
    assert(ans);
    cout << fixed << setprecision(8) << (long double)(ans->first / ans->second) << '\n';
}
```

8.2 非负权无向图最小环与原边方案

问题描述：给定非负权无向图，求至少含 3 个不同顶点的最小权简单环，并给出环上的顶点序列与对应的原边编号。

核心思路：Floyd 求最小环：外层按中间点 k 递增，在用 k 松弛之前，`d[i][j]` 是只经过编号小于 k 的点的最短路。此时枚举 $i<j<k$，若 i,j 都与 k 有边，候选环长为 $d_{ij}+w(i,k)+w(j,k)$；k 是环上编号最大的点，所以每个环都会在其最大点处被考虑到。每轮只记录本轮更优的一对 (i,j)，并立刻沿 `next` 恢复 i 到 j 的路径得到方案（之后 `next` 会被改写）。矩阵 `id` 只保存每对点之间最轻的一条边，自环不进入。

接口契约：MinCycle(n)，n≥0，点 0..n-1。add(u,v,w) 返回从 0 开始的原边号，边数 ≤INT_MAX，0≤w≤LLONG_MAX；允许自环和重边。solve() 从零求至少 3 个不同顶点的简单环，无解 nullopt；故自环忽略，平行边只保留最轻的一条（同权留最早），不计二元环。Result.weight 是 int128，vertices 不重复首点，edges[i] 连接 vertices[i] 与 vertices[(i+1)%size]，边号对应 e；并列返回任意一个，不保证字典序。支持空图、断连、零权、重复 solve 及加边后重算，旧结果独立；e/id 只读。Floyd 更新中间点 k 之前枚举 i<j<k，用 d[i][j]+边 (i,k)+ 边 (j,k) 更新最优值，每轮至多恢复一次路径；严格松弛避免零权环导致方案循环。solve 为 $O(n^3+n^2)$，构造 $O(n^2)$，add 摊还 O(1)，总空间 $O(n^2+m)$，恢复通过 next 逐边走路径，不是模拟 DFS。负边不支持；无穷大 2^120 只作内部哨兵，合法简单路径与候选和均小于 2^95。

```
struct MinCycle
{
    using I = __int128_t;
    struct Edge
    {
        int u, v;
        long long w;
    };
    struct Result
    {
        I weight;
        vector<int> vertices, edges;
    };

    int n;
    vector<Edge> e;
    vector<vector<int>> id;

    MinCycle(int n) : n(n), id(n, vector<int>(n, -1))
    {
        assert(n >= 0);
    }

    int add(int u, int v, long long w)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && w >= 0);
        int k = e.size();
        e.push_back({u, v, w});
        if (u != v && (id[u][v] == -1 || w < e[id[u][v]].w))
            id[u][v] = id[v][u] = k;
        return k;
    }
```

初始化与本轮候选检查（接上页同一结构体）：

```
    optional<Result> solve() const
    {
        const I inf = I(1) << 120;
        vector<vector<I>> d(n, vector<I>(n, inf));
        vector<vector<int>> next(n, vector<int>(n, -1));
        for (int i = 0; i < n; i++)
        {
            d[i][i] = 0;
            for (int j = 0; j < n; j++)
            {
                if (id[i][j] == -1) continue;
                d[i][j] = e[id[i][j]].w;
                next[i][j] = j;
            }
        }
        Result ans{inf, {}, {}};
        for (int k = 0; k < n; k++)
        {
            int u = -1, v = -1;
            for (int i = 0; i < k; i++)
            {
                if (id[i][k] == -1) continue;
                for (int j = i + 1; j < k; j++)
                {
                    if (id[j][k] == -1 || d[i][j] == inf) continue;
                    I w = d[i][j] + e[id[i][k]].w + e[id[j][k]].w;
                    if (w >= ans.weight) continue;
                    ans.weight = w;
                    u = i;
                    v = j;
                }
            }
```

方案恢复与 Floyd 更新（接上页 solve 的 k 循环）：

```
            if (u != -1)
            {
                ans.vertices = {k, u};
                ans.edges = {id[k][u]};
                while (u != v)
                {
                    int x = next[u][v];
                    ans.edges.push_back(id[u][x]);
                    ans.vertices.push_back(x);
                    u = x;
                }
                ans.edges.push_back(id[v][k]);
            }
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (d[i][k] == inf || d[k][j] == inf) continue;
                    I w = d[i][k] + d[k][j];
                    if (w >= d[i][j]) continue;
                    d[i][j] = w;
                    next[i][j] = next[i][k];
                }
            }
        }
        if (ans.weight == inf) return nullopt;
        return ans;
    }
};
```

检查必须在松弛之前 固定环中最大编号点 k，其两侧邻点 i、j 都小于 k。此时 d[i][j] 只允许小于 k 的中间点，与两条原边共同构成合法简单环；先做本轮 Floyd 会使路径经过 k，错误接受重复点。

恢复不能每次改进都做 每轮只记最优 i、j，结束枚举后恢复一次，所有恢复共 $O(n^2)$。next 沿真实路径逐边走，不是模拟 DFS；更新必须严格变小，尤其不能把零权等长路径改成自指。

模板题使用：P6175 无向图的最小环问题

题意：求至少 3 个不同顶点的最小权环；n≤100，m≤5000，边权 1..100000，答案 ≤10000000，可转 long long 输出；无解精确输出 No solution.。

所需模板：MinCycle（第 114 页，8.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：https://www.luogu.com.cn/problem/P6175

```
int main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    MinCycle g(n);
    while (m--)
    {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        g.add(u - 1, v - 1, w);
    }
    auto ans = g.solve();
    if (ans)
        cout << (long long)ans->weight << '\n';
    else
        cout << "No solution.\n";
}
```